Physics-Informed Koopman Neural Operator for Augmented Dynamics Visual Servoing of Multirotors

Archit Krishna Kamath, *Member, IEEE*, Bing Yan, *Member, IEEE*, Peng Shi, *Life Fellow, IEEE*, and Mir Feroskhan, *Member, IEEE*

***Abstract*—This paper introduces a Physics-Informed Koopman Neural Operator (PI-KNO) for augmented dynamics visual servoing of multirotors that integrates Koopman operator theory with neural networks. The proposed method establishes a structured learning framework that effectively captures complex system dynamics while embedding physics-based priors. Unlike fully data-driven approaches, PI-KNO improves generalization and minimizes reliance on extensive real-world training data by employing a hybrid loss function that combines physics-informed constraints with real-time observations. The learned model is incorporated into a monotonically weighted nonlinear model predictive control (NMPC) framework, ensuring precise trajectory tracking while adhering to state and input constraints. Experimental results demonstrate that PI-KNO reduces training time by 16.81% and enhances tracking accuracy by 19.56% compared to conventional Data-Driven Koopman Neural Operators (DD-KNO) and Physics-Informed Neural Networks (PINN). Additionally, under 70.83% uncertainty in camera parameters, PI-KNO achieves 10.6% and 22.2% lower tracking errors than PINN and DD-KNO, respectively. These findings underscore the robustness and efficiency of the proposed approach for real-time multirotor visual servoing applications.**

***Note to Practitioners*—This work presents an implementation oriented view of a learning based predictive controller for aerial robots designed to operate within the limits of onboard computation and sensing. The proposed PI-KNO approach combines physical modeling and data driven learning to create an accurate and stable dynamic predictor suitable for real time deployment. The architecture is distributed between an NVIDIA Jetson and a Pixhawk. The Jetson runs the vision front end, state estimator, PI-KNO predictor, and NMPC optimizer. The estimator fuses visual and inertial data to provide position, velocity, and attitude states, while the NMPC computes high level motion commands at about 30Hz using the PI-KNO rollout. The Pixhawk executes inner rate loops at 250Hz through the native autopilot for attitude stabilization and motor mixing. Communication between the Jetson and Pixhawk uses standard MAVLink setpoints, requiring no firmware modification. The training workflow begins with a nominal visual servoing multirotor model and limited flight data, from which the operator is trained offline using a physics-informed regularizer. At runtime, the estimator updates the state, PI-KNO predicts short horizon dynamics, the NMPC optimizes control sequences, and the Pixhawk executes the commands. This setup achieves reliable tracking across trajectories, maintains physical consistency under uncertainty, and fits within the computational limits of embedded hardware.**

***Index Terms*—Koopman operator, multirotors, neural networks, physics-informed learning, visual servoing.**

Received 4 April 2025; revised 16 October 2025 and 30 December 2025; accepted 28 January 2026. Date of publication 6 February 2026; date of current version 18 February 2026. This article was recommended for publication by Associate Editor J. J. Liang and Editor Z. Li upon evaluation of the reviewers' comments. This work was supported in part by the Ministry of Education, Singapore, through the MOE AcRF Tier 2 February 2024 Grant Call under Grant MOE-T2EP50124-0002; and in part by Australian Research Council under Grant DE250101335. *(Corresponding author: Mir Feroskhan.)*

Archit Krishna Kamath and Mir Feroskhan are with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore 639798 (e-mail: archit.kamath@ntu.edu.sg; mir.feroskhan@ntu.edu.sg).

Bing Yan and Peng Shi are with the School of Electrical and Mechanical Engineering, The University of Adelaide, Adelaide, SA 5005, Australia (e-mail: bing.yan@adelaide.edu.au; peng.shi@adelaide.edu.au).

This article has supplementary downloadable material available at https://doi.org/10.1109/TASE.2026.3661115, provided by the authors.

Digital Object Identifier 10.1109/TASE.2026.3661115

I. Introduction

ACHIEVING high precision control in robotic systems remains a fundamental challenge across a wide range of applications, including industrial automation, autonomous aerial systems, and medical robotics [1]. Attaining this level of precision requires a thorough understanding of the underlying system dynamics, which are frequently nonlinear and complex in nature [2], [3]. For instance, in the case of multirotor platforms performing visual servoing tasks [4], the system behavior is influenced by a combination of aerodynamic forces, sensor uncertainties, and time varying environmental conditions. Visual servoing refers to the use of visual feedback, typically from onboard cameras, to guide the motion of a robot in real time [5]. For multirotors, this involves estimating the position and orientation of objects or features directly from image data, which is often noisy, delayed, and sensitive to environmental variations such as lighting, occlusions, and motion blur [6]. These limitations can severely degrade the accuracy of state estimation and, consequently, the quality of control. As a result, visual servoing presents a uniquely challenging control problem that demands models capable of capturing complex interactions between flight dynamics and perceptual feedback. The motion of the multirotor is determined not only by its inherent flight dynamics but also by the quality of the visual feedback, which introduces complexity.

Model based control strategies, such as proportional–integral–derivative (PID) control [7], linear quadratic regulation (LQR) [8], state feedback control [9], and sliding mode control [10], often depend on simplified assumptions. While effective in structured environments,

1558-3783 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

these approaches typically fail to account for unmodeled uncertainties and dynamic changes, leading to performance degradation in unstructured or unpredictable scenarios [11]. Furthermore, ensuring robustness and stability under such conditions remains a significant obstacle, particularly when some system states are only partially observable [12]. Nonlinear model predictive control (NMPC) has emerged as a powerful framework for managing system constraints and optimizing control actions over a finite horizon [13]. Despite its advantages, NMPC is sensitive to inaccuracies in system modeling and real world uncertainties, which can lead to increased computational demands. To preserve performance, NMPC often requires frequent reoptimization and high fidelity models that necessitate solving complex nonlinear programs at each control step. Efforts to improve robustness, such as the use of chance constraints, min-max optimization [14], tube based methods [15], and adaptive learning mechanisms [16], [17], typically add further computational overhead, making real time deployment increasingly challenging.

Recent progress in machine learning, particularly in deep learning and fuzzy logic [18], [19], has demonstrated strong potential in modeling complex nonlinear dynamics [20], [21]. These methods capture system uncertainties and modeling errors by learning directly from data, rather than relying entirely on first principles. Neural networks have been effectively used to approximate unknown system dynamics [22], compensate for model discrepancies [23], and adapt to time varying environments in real time [24], [25]. When used in conjunction with NMPC, such models improve prediction accuracy and reduce the need for explicit uncertainty handling, thereby easing computational burdens. However, conventional deep learning techniques often lack interpretability, require large volumes of training data, and may exhibit poor generalization outside their training distribution. These issues present significant barriers to real time deployment in safety critical robotic systems [26].

To address these limitations, physics-informed neural networks (PINNs) have been proposed as a hybrid learning approach that incorporates prior physical knowledge into the training process. This reduces data dependency and enforces consistency with governing dynamics [27]. Recent studies have demonstrated the integration of PINNs with NMPC frameworks for controlling quadrotors [26], [28]. In particular, [29] applied PINNs for UAV system estimation and demonstrated their potential to generate physics-consistent models suitable for model-based predictive control. However, despite their promise, PINNs often face challenges related to training inefficiency and hyperparameter sensitivity. These issues limit their scalability to high-dimensional systems exhibiting rapidly evolving dynamics [30].

Koopman neural operators (KNOs) provide a promising alternative by utilizing Koopman operator theory, which transforms nonlinear dynamics into a higher dimensional space where their evolution becomes linear [31]. This approach enables structured and data driven representations of system dynamics, improving both interpretability and computational tractability. KNOs have been applied in modeling fluid flows [32], robotic systems [33], and control tasks. However, conventional KNOs do not enforce physical consistency and are fully reliant on data, making them vulnerable to inaccuracies under sparse or noisy conditions. In light of these challenges, this paper presents the following contributions:

1) This paper introduces PI-KNO, a structured learning framework that integrates Koopman operator theory with neural networks to model multirotor dynamics in visual servoing. By incorporating physics informed constraints, PI-KNO improves generalization and reduces reliance on extensive real world data. Experiments demonstrate a 16.81% reduction in training time and a 19.56% improvement in tracking accuracy relative to data driven KNO and PINN, resulting in improved efficiency and robustness for multirotor control in real time.
2) The learned PI-KNO model is integrated into a monotonically weighted NMPC framework. This ensures robust trajectory tracking while enforcing state and input constraints. The adaptive weighting mechanism dynamically adjusts the prediction horizon, which enhances closed loop stability and reduces computational complexity.
3) The proposed framework is evaluated through extensive simulation and hardware experiments that include benchmarking, ablation, and cross method comparisons. In addition to physics-informed and data driven baselines such as PINN [26] and data driven KNO (DD-KNO) [31], the evaluation encompasses state of the art learning based controller designs KNODE-MPC [34] and SINDy+MPC [35], [36]. The results consistently show that the proposed approach achieves the most accurate trajectory alignment across all evaluated scenarios, yielding a mean percentage soft dynamic-time wrapping (%SDTW) improvement of approximately 14.7% over SINDy+MPC, 24.2% over KNODE-MPC, 40.0% over PINN, and 52.4% over DD-KNO. The framework also achieves a 16.81% reduction in training time and a 19.56% improvement in tracking accuracy relative to existing Koopman and PINN based formulations. Hardware experiments on multirotor visual servoing tasks confirm that the method maintains stable and precise tracking under up to 70.83% uncertainty in camera calibration parameters, consistently outperforming all learning based controllers in convergence speed, spatial temporal coherence and robustness to uncertainties.

The remainder of the paper is organized as follows. Section II presents the mathematical preliminaries on Koopman operator theory and physics-informed learning. Section III describes the nominal visual servoing multirotor dynamics and the PI-KNO modeling process. The control strategy, based on monotonically weighted NMPC, is detailed in Section IV followed by the overall framework in Section V. Section VI discusses the experimental results, and Section VII concludes the paper.

II. Mathematical Preliminaries

A. Koopman Neural Operator

The Koopman Neural Operator (KNO) [31] integrates Koopman operator theory with neural networks to develop a

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

framework for learning nonlinear system dynamics, facilitating the explicit identification of system equations. To illustrate its functionality, consider a discrete-time nonlinear dynamical system represented by:

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) \tag{1}$$

where $\mathbf{x}_i \in \mathbb{R}^n$ represents the system states, and $\mathbf{u}_i \in \mathbb{R}^p$ denotes the control inputs at the ith sampling instance. The function $\mathbf{f}(\cdot,\cdot) : \mathbb{R}^{m+n} \to \mathbb{R}^n$ defines the system dynamics and is assumed to be Lipschitz continuous before discretization. The main objective of KNO is to approximate the evolution of system states, represented in (1), using a neural network based on the Koopman operator theory which is defined as follows:

Definition 1: For an observable function $\mathbf{F} : \mathbb{R}^{m+n} \to \mathbb{C}$, the Koopman operator $\mathcal{K}$ is defined as:

$$\mathcal{K}\mathbf{F}(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{F}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}) = \mathbf{F}(\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k), \mathbf{u}_{k+1}) \tag{2}$$

This implies that the operator $\mathcal{K}$ captures the evolution of the observable function from the kth time sample to the $(k+1)^{th}$ time sample.

As stated in Definition 1, traditional Koopman operators act on an infinite-dimensional observation function space, i.e., $\mathbb{C}$, which is not suitable for practical applications. To overcome this limitation, the Koopman operator must be approximated in a finite-dimensional space. This is achieved by representing the observable function $\mathbf{F}$ as a neural network parameterized by the weight vector $\mathbf{w}$. The function is expressed as $\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}) : \mathbb{R}^{m+n} \to \mathbb{R}^d$ where d represents the number of basis functions. Mathematically, this can be written as:

$$\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}) = \left[\varphi_1(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}_1) \cdots \varphi_d(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}_d)\right]^T \tag{3}$$

where $\varphi_i(\cdot,\cdot;\mathbf{w}_i)$ denotes the basis function and will be elaborated later in this section.

This formulation transforms the mapping in Definition 1 into the following finite-dimensional approximation:

$$\mathcal{K}\mathbf{F}(\mathbf{x}_k, \mathbf{u}_k) \approx \Gamma\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}) \tag{4}$$

where $\Gamma \in \mathbb{R}^{d\times d}$ is a finite-dimensional linear mapping matrix that must be learned alongside the weights $\mathbf{w}$. To validate the accuracy of this approximation, one must determine whether there exist values for the parameters $\mathbf{w}$ and Γ that satisfy the following condition:

$$\|\mathcal{K}\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}) - \Gamma\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w})\| \le \delta \tag{5}$$

where $\|\cdot\|$ denotes the function norm, and $\delta \in \mathbb{R}^+$ represents a small tolerance value. To prove the validity of the expression (5), the universal approximation theorem [37] is considered which states that for any continuous function $\mathbf{F}(\mathbf{x}_k, \mathbf{u}_k)$ there exists a feed-forward neural network of sufficient depth, represented as $\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w})$, such that:

$$\|\mathbf{F}(\mathbf{x}_k, \mathbf{u}_k) - \mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w})\| \le \epsilon \tag{6}$$

where $\epsilon \in \mathbb{R}^+$ represents a small tolerance value. Rewriting the inequality given in (5) using the triangle inequality property, we obtain:

$$\begin{aligned}\|\mathcal{K}\mathbf{\Phi} - \Gamma\mathbf{\Phi}\| &= \|\mathcal{K}\mathbf{\Phi} + \mathcal{K}\mathbf{F} - \mathcal{K}\mathbf{F} + \Gamma\mathbf{F} - \Gamma\mathbf{F} - \Gamma\mathbf{\Phi}\| \\ &\le \|\mathcal{K}\mathbf{\Phi} - \mathcal{K}\mathbf{F}\| + \|\mathcal{K}\mathbf{F} - \Gamma\mathbf{F}\| + \|\Gamma\mathbf{F} - \Gamma\mathbf{\Phi}\| \\ &\le \|\mathcal{K}\|\epsilon + \|\mathcal{K}\mathbf{F} - \Gamma\mathbf{F}\| + \|\Gamma\|\epsilon \end{aligned} \tag{7}$$

where $\mathbf{\Phi}$ and $\mathbf{F}$ are abbreviations for $\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w})$ and $\mathbf{F}(\mathbf{x}_k, \mathbf{u}_k)$. To satisfy (5), the value of ϵ is chosen as $\epsilon = \frac{\delta}{2(\|\mathcal{K}\|+\|\Gamma\|)}$. Substituting this into (7), we obtain:

$$\|\mathcal{K}\mathbf{\Phi} - \Gamma\mathbf{\Phi}\| \le \|\mathcal{K}\mathbf{F} - \Gamma\mathbf{F}\| + \frac{\delta}{2} \tag{8}$$

Since $\mathbf{F}$ is a continuous observable function that, according to the Universal Approximation Theorem, can be closely approximated by a neural network, the matrix Γ is selected as the best finite-dimensional approximation of $\mathcal{K}$ within the subspace spanned by $\mathbf{F}$, ensuring that $\|\mathcal{K}\mathbf{F} - \Gamma\mathbf{F}\| \le \frac{\delta}{2}$. Substituting this into (8), the final inequality simplifies to:

$$\|\mathcal{K}\mathbf{\Phi} - \Gamma\mathbf{\Phi}\| \le \delta \tag{9}$$

This result implies that there exists a neural network $\mathbf{\Phi}$ parameterized by $\mathbf{w}$ and a linear mapping matrix Γ that can approximate the Koopman operator $\mathcal{K}$. Consequently, the state evolution described by the nonlinear dynamics in (1) can be approximated using the finite-dimensional Koopman framework, given by:

$$\mathbf{x}_{k+1} = \Gamma\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}) = \Gamma \begin{bmatrix} \varphi_1(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}_1) \\ \vdots \\ \varphi_d(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}_d) \end{bmatrix} \tag{10}$$

Remark 1: The dimension of the finite-dimensional linear mapping matrix Γ can be altered to $\mathbb{R}^{n\times d}$ in order to map the state evolution $\mathbf{x}_{k+1}$ to $\Gamma\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w})$ while still maintaining $\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}) : \mathbb{R}^{m+n} \to \mathbb{R}^d$.

A crucial aspect of the KNO framework is the choice of basis functions φ_i, which are parameterized by the weight vector $\mathbf{w}_i$. Common approaches include classic feed-forward neural networks, recurrent neural networks (RNNs), graph neural networks, and convolutional neural networks (CNNs). However, in this study, we employ a quadratic polynomial basis function for each φ_i, expressed as:

$$\varphi_i(\mathbf{x}, \mathbf{u}; \mathbf{w}_i) = w_i^0 + \|\mathbf{w}_i^{\mathbf{x}}\mathbf{x}\mathbf{x}^T\| + \|\mathbf{w}_i^{\mathbf{u}}\mathbf{u}\mathbf{u}^T\| + \|\mathbf{w}_i^{\mathbf{x},\mathbf{u}}\mathbf{x}\mathbf{u}^T\| \tag{11}$$

where $w_i^0 \in \mathbb{R}$, $\mathbf{w}_i^{\mathbf{x}} \in \mathbb{R}^{n\times n}$, $\mathbf{w}_i^{\mathbf{u}} \in \mathbb{R}^{m\times m}$, and $\mathbf{w}_i^{\mathbf{x},\mathbf{u}} \in \mathbb{R}^{m\times n}$ are the trainable weight matrices of the regression function. The matrices $\mathbf{w}_i^{\mathbf{x}}$, $\mathbf{w}_i^{\mathbf{u}}$, $\mathbf{w}_i^{\mathbf{x},\mathbf{u}}$, and the scalar term w_i^0 are collectively denoted as $\mathbf{w}_i$ and will be used as such throughout the remainder of this paper. This basis function selection enhances interpretability and facilitates explicit identification of the system dynamics described in (1). To estimate the linear mapping matrix Γ, the following objective function can be used as follows:

$$\Gamma = \arg\min_{\Gamma} \sum_{k=1}^{N} \|\Gamma\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}) - \mathbf{x}_{k+1}\|^2 \tag{12}$$

where N denotes the observation set comprising pairs $(\mathbf{x}_k, \mathbf{u}_k)$ for all $k = 1, 2, \ldots, N$. The parameter $\mathbf{w}$ can be estimated using the following objective function:

$$\mathbf{w} = \arg\min_{\mathbf{w}} \sum_{k=1}^{N} \left\|\mathbf{\Phi}(\mathbf{x}_k, \mathbf{u}_k; \mathbf{w}) - \Gamma^+\mathbf{x}_{k+1}\right\|^2 \tag{13}$$

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

where $^+$ denotes the pseudo-inverse, also known as the Moore-Penrose inverse, of the matrix Γ whose dimensions must now be altered to $n \times d$ following Remark 1. The minimization objectives in (12) and (13) can be optimized using the Adam algorithm to determine the values of Γ and $\mathbf{w}$. It is important to note that the value of N must be sufficiently large to ensure that the dynamics described by (1) are accurately captured. Moreover, the current Koopman approximation obtained from the KNO does not explicitly account for system uncertainties and modeling inaccuracies inherent in practical nonlinear systems. To address these limitations, the conventional training approach for the KNO must be replaced with a physics-informed training algorithm, which is detailed next.

B. Physics-Informed Learning for KNO

The training of the conventional KNO is currently based on estimating Γ using (12) and $\mathbf{w}$ using (13), which heavily relies on the availability of observation samples of the system states and control inputs. This approach entirely disregards the physics-based knowledge of the system dynamics, commonly referred to as the nominal dynamics, which represent the known components of $\mathbf{f}(\cdot,\cdot)$ and is represented as $\mathbf{f}_{\text{nom}}(\cdot,\cdot)$. As a result, the method remains fully data-driven, increasing the required number of samples N to achieve an accurate approximation. To address this limitation, the physics-informed learning framework modifies (12) to incorporate two distinct loss terms: the physics loss and the real-data loss. These terms account for parametric uncertainties and modeling inaccuracies, respectively, enhancing the robustness of the Koopman approximation. The physics loss $\mathcal{L}_p$ is defined as:

$$\mathcal{L}_p = \frac{1}{N_p}\sum_{i\in\mathcal{P}} \|\Gamma\mathbf{\Phi}(\mathbf{w}) - (\mathbf{f}_{\text{nom}} + \mathbf{f}_{\text{phy}})\|^2\Big|_{(\mathbf{x}_i,\mathbf{u}_i)} \tag{14}$$

where $\mathcal{P}$ denotes the physics loss dataset, which is generated by treating $\mathbf{f}_{\text{phy}}$ as an additive parametric uncertainty sampled from a predefined probability distribution, such as a normal distribution parameterized by $\mathcal{N}(\mu,\sigma)$, where μ and σ represent the mean and standard deviation, respectively. The term N_p represents the number of samples in the dataset $\mathcal{P}$.

The modeling inaccuracies are typically accounted for by using a real-data loss term $\mathcal{L}_r$ which is defined as:

$$\mathcal{L}_r = \frac{1}{N_r}\sum_{j\in\mathcal{D}} \|\Gamma\mathbf{\Phi}(\mathbf{w}) - \mathbf{y}_j\|^2\Big|_{\mathbf{y}_j=\mathbf{x}_{j+1}} \tag{15}$$

where $\mathcal{D}$ represents the real-time dataset which is obtained by conducting real-time experiments on the dynamical system. Typically, a Proportional-Integral-Derivative (PID) controller is used to stabilize the nonlinear system to conduct these experiments and the data from these trails is recorded in the dataset $\mathcal{D}$. N_r represents the size of $\mathcal{D}$. The total composite loss for the training the KNO is now given as $\mathcal{L}_p + \mathcal{L}_r$ which will be used to obtain Γ as follows:

$$\Gamma = \arg\min_{\Gamma}\left(\mathcal{L}_p + \mathcal{L}_r\right) \tag{16}$$

The conventional KNO, when trained using the algorithm presented in (9), is referred to as the Physics-Informed Koopman Neural Operator (PI-KNO) and will be denoted as PI-KNO henceforth. Following the logic presented to derive the updates for $\mathbf{w}$, the physics-informed learning for $\mathbf{w}$ is:

$$\mathbf{w} = \arg\min_{\mathbf{w}}\left(\bar{\mathcal{L}}_p + \bar{\mathcal{L}}_r\right) \tag{17}$$

where the terms $\bar{\mathcal{L}}_p$ and $\bar{\mathcal{L}}_r$ are given as:

$$\begin{aligned}\bar{\mathcal{L}}_p &= \frac{1}{N_p}\sum_{i\in\mathcal{P}} \|\mathbf{\Phi}(\mathbf{w}) - \Gamma^+(\mathbf{f}_{\text{nom}} + \mathbf{f}_{\text{phy}})\|^2\Big|_{(\mathbf{x}_i,\mathbf{u}_i)}\\ \bar{\mathcal{L}}_r &= \frac{1}{N_r}\sum_{j\in\mathcal{D}} \|\mathbf{\Phi}(\mathbf{w}) - \Gamma^+\mathbf{y}_j\|^2\Big|_{\mathbf{y}_j=\mathbf{x}_{j+1}}\end{aligned} \tag{18}$$

The training algorithmic flow for the PI-KNO can be found in the supplementary material attached to this paper.

III. System Dynamics Modeling

A. Nominal Model

This study utilizes the nominal dynamics of the augmented dynamics visual servoing model for a multirotor, as presented in [26]. The formulation begins by considering the 6-DoF dynamical model of the multirotor, expressed as follows:

$$\begin{bmatrix}\ddot{\boldsymbol{\eta}}_o\\ \ddot{\boldsymbol{\eta}}_i\end{bmatrix} = \begin{bmatrix}\bar{\mathbf{f}}_o\\ \bar{\mathbf{f}}_i\end{bmatrix} + \begin{bmatrix}\mathbf{J}_o & \mathbf{0}\\ \mathbf{0} & \mathbf{J}_i\end{bmatrix}\begin{bmatrix}\bar{\mathbf{u}}_o\\ \bar{\mathbf{u}}_i\end{bmatrix} + \begin{bmatrix}\mathbf{D}_o\\ \mathbf{D}_i\end{bmatrix} \tag{19}$$

The state variables of the multirotor are represented as outer-loop position and altitude, $\boldsymbol{\eta}_o \in \mathbb{R}^3 = [x\ y\ z]$, and inner-loop attitude (roll, pitch, and yaw), $\boldsymbol{\eta}_i \in \mathbb{R}^3 = [\phi\ \theta\ \psi]$. The nonlinear dynamics are captured by $\bar{\mathbf{f}}_o \in \mathbb{R}^3 = [0\ 0\ -g]^T$ and $\bar{\mathbf{f}}_i \in \mathbb{R}^3 = [a_1\dot{\theta}\dot{\psi}\quad a_2\dot{\psi}\dot{\phi}\quad a_3\dot{\phi}\dot{\theta}]^T$, along with inertia matrices $\mathbf{J}_o \in \mathbb{R}^{3\times3} = \mathbf{diag}\left[\frac{1}{m}\quad\frac{1}{m}\quad\frac{1}{m}\right]$ and $\mathbf{J}_i \in \mathbb{R}^{3\times3} = \mathbf{diag}\left[\frac{1}{J_x}\quad\frac{1}{J_y}\quad\frac{1}{J_z}\right]$. Here, g, m, and J_x, J_y, J_z denote the acceleration due to gravity, multirotor mass, and moments of inertia along the respective axes. The constants a_1, a_2, and a_3 are given by $a_1 = \frac{J_y - J_z}{J_x}$, $a_2 = \frac{J_z - J_x}{J_y}$, and $a_3 = \frac{J_x - J_y}{J_z}$. The outer-loop auxiliary control effort and inner-loop actual control efforts are represented by $\bar{\mathbf{u}}_o \in \mathbb{R}^3 = [u_x\quad u_y\quad u_z]^T$ and $\bar{\mathbf{u}}_i \in \mathbb{R}^3 = [u_\phi\quad u_\theta\quad u_\psi]^T$, respectively. Lastly, the bounded disturbances in the outer and inner loops are denoted by $\mathbf{D}_o \in \mathbb{R}^3$ and $\mathbf{D}_i \in \mathbb{R}^3$, respectively.

The augmented dynamics visual servoing model is derived by integrating the outer-loop multirotor dynamics with the acceleration-based image dynamics, expressed as follows:

$$\ddot{\mathbf{r}} = \mathbf{\Lambda}\bar{\mathbf{A}} + \begin{bmatrix}\bar{\mathbf{V}}^T\Delta_1\bar{\mathbf{V}}\\ \bar{\mathbf{V}}^T\Xi_1\bar{\mathbf{V}}\\ \vdots\\ \bar{\mathbf{V}}^T\Delta_n\bar{\mathbf{V}}\\ \bar{\mathbf{V}}^T\Xi_n\bar{\mathbf{V}}\end{bmatrix} + \begin{bmatrix}\mathbf{D}_{\mathbf{r}}^{\mathbf{1}}\\ \vdots\\ \mathbf{D}_{\mathbf{r}}^{\mathbf{n}}\end{bmatrix} \tag{20}$$

where n denotes the number of points of interest (POIs) in the camera plane, and $\ddot{\mathbf{r}} \in \mathbb{R}^{2n}$ represents the acceleration of these POIs in the image space. The matrix $\mathbf{\Lambda} \in \mathbb{R}^{2n\times6}$ corresponds to the stacked camera interaction matrix for the n POIs. The vector $\bar{\mathbf{A}} \in \mathbb{R}^6$ is defined as $\bar{\mathbf{A}} = \left[\mathbf{a}\ \boldsymbol{\alpha}\right]^T$, where $\mathbf{a} \in \mathbb{R}^3$ and $\boldsymbol{\alpha} \in \mathbb{R}^3$ represent the 3D linear and angular accelerations of the POIs, respectively. Similarly, the velocity vector $\bar{\mathbf{V}} \in \mathbb{R}^6$ is given by $\bar{\mathbf{V}} = \left[\boldsymbol{\nu}\ \boldsymbol{\omega}\right]^T$, where $\boldsymbol{\nu} \in \mathbb{R}^3$ and $\boldsymbol{\omega} \in \mathbb{R}^3$ denote the corresponding 3D linear and angular velocities. The POIs are represented in the image plane as $\mathbf{r} = [u_1\ v_1\ u_2\ v_2\ \cdots\ u_n\ v_n]^T$.

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

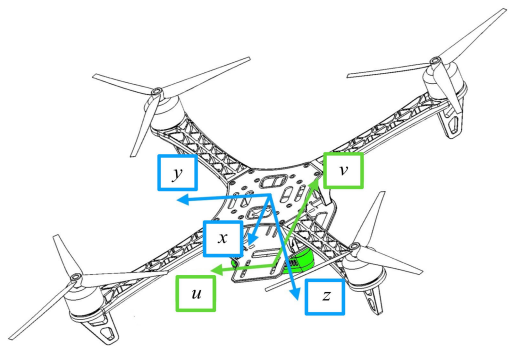


Fig. 1. Multirotor and camera frames of reference.

The matrices $\Delta_i \in \mathbb{R}^{6\times 6}$ and $\Xi_i \in \mathbb{R}^{6\times 6}$ are obtained as described in [26] and are functions of the image coordinates $\mathbf{r}$, the focal length $f \in \mathbb{R}^+$, and the depth of the ith POI, denoted by $\mathbf{P}_{z_i} \in \mathbb{R}^+$. Additionally, $\mathbf{D}^i_{\mathbf{r}} \in \mathbb{R}^2$ represents the bounded image disturbance vector associated with the ith POI.

To derive the augmented dynamical model, the outer-loop dynamics from (19) must be integrated with the pixel acceleration dynamics from (20). However, since the pixel acceleration in (20) is defined for 2D pixel coordinates, whereas the outer-loop dynamics of the multirotor involve three variables, their relationship must first be established. This is achieved by decomposing the outer-loop dynamics into two subsystems, as follows:

$$\begin{bmatrix} \ddot{\eta}_{dep} \\ \ddot{\boldsymbol{\eta}}_{pos} \end{bmatrix} = \begin{bmatrix} \bar{f}_{dep} \\ \bar{\mathbf{f}}_{pos} \end{bmatrix} + \begin{bmatrix} J_{dep} & \mathbf{0} \\ \mathbf{0} & \mathbf{J}_{pos} \end{bmatrix} \begin{bmatrix} \bar{u}_{dep} \\ \bar{\mathbf{u}}_{pos} \end{bmatrix} + \begin{bmatrix} D_{dep} \\ \mathbf{D}_{pos} \end{bmatrix} \tag{21}$$

where the subscripts dep and pos correspond to the depth and position subsystems, respectively. For example, in a camera setup as depicted in Fig. 1, the dep component aligns with the multirotor's z-axis, while the components of $\boldsymbol{\eta}_{pos} \in \mathbb{R}^2$ correspond to the x- and y-axes of the multirotor.

The augmented dynamical model is then obtained by combining the pos subsystem with (20). To establish the relationship between the variables in (20) and those in (21), a rotation matrix $\mathbf{R}^{UAV}_{cam} \in \mathbb{R}^{2\times 2}$ is introduced to transform the variables from the multirotor's body frame to the camera frame. The transformed velocity and acceleration terms, given by $\mathbf{R}^{UAV}_{cam}\dot{\boldsymbol{\eta}}_{pos} \in \mathbb{R}^2$ and $\mathbf{R}^{UAV}_{cam}\ddot{\boldsymbol{\eta}}_{pos} \in \mathbb{R}^2$, are related to $\bar{\mathbf{V}}$ and $\bar{\mathbf{A}}$ as follows:

$$\begin{aligned} \bar{\mathbf{V}} &= \left[\mathbf{R}^{UAV}_{cam}\dot{\boldsymbol{\eta}}_{pos} \;\; 0 \;\; 0 \;\; 0\right]^T, \\ \bar{\mathbf{A}} &= \left[\mathbf{R}^{UAV}_{cam}\ddot{\boldsymbol{\eta}}_{pos} \;\; 0 \;\; 0 \;\; 0\right]^T. \end{aligned} \tag{22}$$

Substituting (22) into (20) and eliminating the zero columns, the augmented outer-loop dynamics of the multirotor is obtained as:

$$\begin{bmatrix} \ddot{\eta}_{dep} \\ \ddot{\mathbf{r}} \end{bmatrix} = \begin{bmatrix} \bar{f}_{dep} \\ \mathbf{F}_{aug} \end{bmatrix} + \begin{bmatrix} J_{dep} & \mathbf{0}_{2n\times 1} \\ \mathbf{0} & \mathbf{J}_{aug} \end{bmatrix} \begin{bmatrix} \bar{u}_{dep} \\ \bar{\mathbf{u}}_{pos} \end{bmatrix} + \begin{bmatrix} D_{dep} \\ \mathbf{D}_{aug} \end{bmatrix} \tag{23}$$

where $\mathbf{F}_{aug} \in \mathbb{R}^{2n}$, $\mathbf{J}_{aug} \in \mathbb{R}^{2n\times 2}$, and $\mathbf{D}_{aug} \in \mathbb{R}^{2n}$ (the transformed bounded disturbance vector) are functions of $\mathbf{r}$, $\dot{\mathbf{r}}$, $\boldsymbol{\eta}_{pos}$, and $\dot{\boldsymbol{\eta}}_{pos}$. The complete state-space representation for the augmented dynamics visual servoing of the multirotor is then given by:

$$\underbrace{\begin{bmatrix} \ddot{\eta}_{dep} \\ \ddot{\mathbf{r}} \\ \ddot{\boldsymbol{\eta}}_i \end{bmatrix}}_{\dot{\boldsymbol{\eta}}_{\text{ADVS}}} = \underbrace{\begin{bmatrix} \bar{f}_{dep} \\ \mathbf{F}_{aug} \\ \bar{\mathbf{f}}_i \end{bmatrix} + \begin{bmatrix} J_{dep} \\ \mathbf{J}_{aug} \\ \mathbf{J}_i \end{bmatrix} \begin{bmatrix} \bar{u}_{dep} \\ \bar{\mathbf{u}}_{pos} \\ \bar{\mathbf{u}}_i \end{bmatrix}}_{\mathbf{f}_{N-\text{ADVS}}(\eta_{dep},\mathbf{r},\boldsymbol{\eta}_i,\bar{u}_{dep},\bar{\mathbf{u}}_{pos},\bar{\mathbf{u}}_i)} + \underbrace{\begin{bmatrix} D_{dep} \\ \mathbf{D}_{aug} \\ \mathbf{D}_i \end{bmatrix}}_{\mathbf{D}_{\text{ADVS}}}$$

$$\implies \dot{\boldsymbol{\eta}}_{\text{ADVS}} = \mathbf{f}_{\text{ADVS}}(\eta_{dep}, \mathbf{r}, \boldsymbol{\eta}_i, \bar{u}_{dep}, \bar{\mathbf{u}}_{pos}, \bar{\mathbf{u}}_i) \tag{24}$$

where $\dot{\boldsymbol{\eta}}_{\text{ADVS}} \in \mathbb{R}^{4n+8}$ represents the overall state vector of the augmented dynamics model. The term $\mathbf{f}_{N-\text{ADVS}} \in \mathbb{R}^{4n+8}$ corresponds to the nominal model of the augmented dynamics, while $\mathbf{D}_{\text{ADVS}} \in \mathbb{R}^{4n+8}$ represents the aggregated disturbance term. This disturbance term encompasses system uncertainties, image noise, and modeling inaccuracies. Finally, $\mathbf{f}_{\text{ADVS}}(\eta_{dep}, \mathbf{r}, \boldsymbol{\eta}_i, \bar{u}_{dep}, \bar{\mathbf{u}}_{pos}, \bar{\mathbf{u}}_i) \in \mathbb{R}^{4n+8}$ defines the complete nonlinear state-space representation of the augmented dynamics visual servoing approach. For conciseness, the state-space representation in (24) is reformulated as:

$$\dot{\boldsymbol{\eta}}_{\text{ADVS}} = \mathbf{f}_{\text{ADVS}}(\boldsymbol{\eta}_{\text{ADVS}}, \bar{\mathbf{u}}) \tag{25}$$

The following section introduces the PI-KNO framework for modeling the dynamics in (25), where the nominal dynamics are given by $\mathbf{f}_{N-\text{ADVS}}(\boldsymbol{\eta}_{\text{ADVS}}, \bar{\mathbf{u}})$.

Remark 2: The transformed bounded disturbance vector i.e. $\mathbf{D}_{aug}$ combines the system disturbance $\mathbf{D}_{pos}$ and the bounded image noise $\mathbf{D}^{\mathbf{i}}_{\mathbf{r}}$ for each POI being tracked. This merger facilitates in designing of a single controller to tackle both.

B. PI-KNO Based Dynamic Modeling

To apply the PI-KNO framework for modeling the dynamics described in (25), the continuous-time system must be discretized. This process can be interpreted as solving an initial value problem, given that the state variables η_{dep}, $\mathbf{r}$, and $\boldsymbol{\eta}_i$ are known at $t = 0$. The solution to this problem, assuming a sampling interval of T, is given by:

$$\begin{aligned} &\int_{t=k}^{t=k+1} d\boldsymbol{\eta}_{\text{ADVS}} = \int_{t=0}^{t=T} \mathbf{f}_{\text{ADVS}} dt \\ &\implies \boldsymbol{\eta}_{\text{ADVS}}[k+1] = \boldsymbol{\eta}_{\text{ADVS}}[k] + \int_{t=0}^{t=T} \mathbf{f}_{\text{ADVS}} dt \\ &\implies \boldsymbol{\eta}_{\text{ADVS}}[k+1] = \mathbf{F}_{\text{ADVS}}(T, \boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}[k]) \\ &\implies \boldsymbol{\eta}_{\text{ADVS}}[k+1] = \mathbf{F}_{\text{ADVS}}(\boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}[k]) \end{aligned} \tag{26}$$

where $\boldsymbol{\eta}_{\text{ADVS}}[k] = [\eta_{dep}[k] \;\; \mathbf{r}[k] \;\; \boldsymbol{\eta}_i[k]]$ and $\bar{\mathbf{u}}[k] = [\bar{u}_{dep}[k] \;\; \bar{\mathbf{u}}_{pos}[k] \;\; \bar{\mathbf{u}}_i[k]]$. The objective now will be to develop a KNO model to approximate the dynamics presented in (26) which can be considered as follows:

$$\begin{aligned} \boldsymbol{\eta}_{\text{ADVS}}[k+1] &= \mathbf{F}_{\text{ADVS}}(\boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}[k]) \\ &\approx \Gamma_{\text{ADVS}} \boldsymbol{\Phi}_{\text{ADVS}}(\boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}[k]; \mathbf{w}_{\text{ADVS}}) \end{aligned} \tag{27}$$

where $\Gamma_{\text{ADVS}} \in \mathbb{R}^{4n+8\times d}$, with d denoting the number of basis functions used. The influence of varying d on accuracy and training overhead is analyzed and discussed in Section VI.

Remark 3: In practical applications, the dynamics governing the actual visual servoing multirotor, denoted as $\mathbf{F}_{\text{ADVS}}$, inherently include system uncertainties in mass, inertia, and camera focal length, along with modeling inaccuracies. Consequently,

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

during simulations, only the nominal system dynamics can be known, which, in this case, are represented by $\mathbf{F}_{N-\text{ADVS}}$.

The network representation is given by $\mathbf{\Phi}_{\text{ADVS}} = [\varphi_i(\boldsymbol{\eta}_{\text{ADVS}}, \bar{\mathbf{u}}; \mathbf{w}_i)]^T, \quad \forall\ i = 1, 2, \ldots, d$ where each basis function is formulated using a quadratic polynomial representation, as follows:

$$\varphi_i = w_i^0 + \|\mathbf{w}_i^{\boldsymbol{\eta}_{\text{ADVS}}} \boldsymbol{\eta}_{\text{ADVS}} \boldsymbol{\eta}_{\text{ADVS}}^T\| + \|\mathbf{w}_i^{\bar{\mathbf{u}}} \bar{\mathbf{u}} \bar{\mathbf{u}}^T\| + \|\mathbf{w}_i^{\boldsymbol{\eta}_{\text{ADVS}}, \bar{\mathbf{u}}} \boldsymbol{\eta}_{\text{ADVS}} \bar{\mathbf{u}}^T\| \tag{28}$$

where $w_i^0 \in \mathbb{R}$, $\mathbf{w}_i^{\boldsymbol{\eta}_{\text{ADVS}}} \in \mathbb{R}^{4n+8 \times 4n+8}$, $\mathbf{w}_i^{\bar{\mathbf{u}}} \in \mathbb{R}^{6\times 6}$, and $\mathbf{w}_i^{\boldsymbol{\eta}_{\text{ADVS}}, \bar{\mathbf{u}}} \in \mathbb{R}^{6 \times 4n+8}$.

To estimate the parameter Γ_{ADVS} and $\mathbf{w}_{\text{ADVS}} = \mathbf{w}_i\ \forall\ i = 1, 2, \ldots, d$, optimization is performed on a dataset $\mathcal{N}$, which consists of observation pairs $(\boldsymbol{\eta}_{\text{ADVS}}, \bar{\mathbf{u}})$ collected from dynamic simulations and real-time hardware experiments. The dynamic simulations represent the physics-loss dataset $\mathcal{P}$, while the real-time experiments constitute the real-loss dataset $\mathcal{D}$. It is important to note that the real experiments are conducted with a PID controller to stabilize the multirotor during flight. As previously discussed, the primary objective of the dynamic simulations is to model system uncertainties and incorporate them into the physics-loss term $\mathcal{L}_p$, which is defined as:

$$\mathcal{L}_p = \frac{1}{N_p} \sum_{i \in \mathcal{P}} \left\| \Gamma_{\text{ADVS}} \mathbf{\Phi}_{\text{ADVS}}(\mathbf{w}_{\text{ADVS}}) - (\mathbf{F}_{N-\text{ADVS}} + \mathbf{F}_{P-\text{ADVS}}) \right\|^2 \Big|_{(\boldsymbol{\eta}_{\text{ADVS}}[i], \bar{\mathbf{u}}[i])} \tag{29}$$

where the combination $(\mathbf{F}_{N-\text{ADVS}} + \mathbf{F}_{P-\text{ADVS}})$ represents the output of simulating the nominal dynamics of the system and considering uncertainties in system parameters such as the mass m, the inertial values J_x, J_y and J_z and the focal length f to have been samples from a normal distribution [26].

To account for modeling inaccuracies, the real-data loss term $\mathcal{L}_r$ is introduced, defined as:

$$\mathcal{L}_r = \frac{1}{N_r} \sum_{j \in \mathcal{D}} \left\| \Gamma_{\text{ADVS}} \mathbf{\Phi}_{\text{ADVS}}(\mathbf{w}_{\text{ADVS}}) - \mathbf{y}_{\text{ADVS}}[j] \right\|^2 \tag{30}$$

where $\mathbf{y}_{\text{ADVS}}$ denotes the outputs obtained from real-flight experiments using a PID controller. The total loss function $\mathcal{L}_p + \mathcal{L}_r$ is then used to estimate the value of Γ_{ADVS} using the following expression:

$$\Gamma_{\text{ADVS}} = \min_{\Gamma_{\text{ADVS}}} \left(\mathcal{L}_p + \mathcal{L}_r \right) \tag{31}$$

The estimated values of Γ_{ADVS} are then utilized to update $\mathbf{w}_{\text{ADVS}}$ by minimizing the following optimization objective:

$$\mathbf{w}_{\text{ADVS}} = \arg \min_{\mathbf{w}_{\text{ADVS}}} \frac{1}{N_r} \sum_{j \in \mathcal{D}} \left\| \mathbf{\Phi}_{\text{ADVS}}(\mathbf{w}_{\text{ADVS}}) - \Gamma_{\text{ADVS}}^{+} \mathbf{y}_{\text{ADVS}}[j] \right\|^2 \tag{32}$$

The dataset $\mathcal{D}$ is used for estimating $\mathbf{w}_{\text{ADVS}}$ to ensure accuracy; however, a combination of $\mathcal{D}$ and $\mathcal{P}$ can also be employed for this estimation, as discussed in Remark 2. Collectively, this would lead to the development of a fully trained PI-KNO representation for the augmented-dynamics visual servoing multirotor model which is represented as:

$$\boldsymbol{\eta}_{\text{ADVS}}[k+1] = \Gamma_{\text{ADVS}} \mathbf{\Phi}_{\text{ADVS}}(\boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}[k]; \mathbf{w}_{\text{ADVS}}) \tag{33}$$

The following section outlines the development of the control law based on nonlinear model predictive control, utilizing the learned discrete-time model presented in (33).

IV. Control Design

The motivation for employing NMPC to control the augmented dynamics visual servoing multirotor is not only to ensure system stability but also to enforce adherence to prescribed input and state constraints. Input constraints in the control design help mitigate the risk of actuator saturation, while state constraints prevent issues such as target loss. This is particularly important as the image pixel-related term, denoted as $\mathbf{r}$, is included among the states in $\boldsymbol{\eta}_{\text{ADVS}}$.

However, conventional NMPC formulations often demand significant computational resources, making them impractical for real-time implementation on multirotors with limited onboard computing capabilities, especially when accounting for parametric uncertainties and modeling inaccuracies. The proposed PI-KNO model addresses these challenges by providing a reliable estimate that effectively compensates for uncertainties and inaccuracies, improving the model's predictive accuracy. Nevertheless, the challenge of incorporating terminal constraints within the NMPC formulation to guarantee stability persists, still contributing to the computational burden. A solution for this is to formulate a monotonically weighted NMPC based on the PI-KNO dynamics as follows:

$$\min_{\bar{\mathbf{u}}[k]} J(\boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}[k]) = \min_{\bar{\mathbf{u}}[k]} \sum_{i=1}^{T} \left(\frac{i}{T}\right)^p (\|\boldsymbol{\eta}_{\text{ADVS}}^d[k+i] - \boldsymbol{\eta}_{\text{ADVS}}[k+i]\|_Q^2 + \|\bar{\mathbf{u}}[k+i]\|_R^2)$$
$$\text{Subject to: } \begin{cases} \boldsymbol{\eta}_{\text{ADVS}}[k+1] = \Gamma_{\text{ADVS}} \mathbf{\Phi}_{\text{ADVS}}(\mathbf{w}_{\text{ADVS}}) \\ \forall\ \boldsymbol{\eta}_{\text{ADVS}}[k] \in \mathcal{X} \\ \forall\ \bar{\mathbf{u}}[k] \in \mathcal{U} \\ \forall\ i = 1, 2, \cdots, T \end{cases} \tag{34}$$

where $p \in \mathbb{N}$ is an integer, $\boldsymbol{\eta}_{\text{ADVS}}^d[k]$ denotes the desired system state values $\boldsymbol{\eta}_{\text{ADVS}}[k]$ at the kth time step, and the weighted norms are defined as $\|\boldsymbol{\eta}_{\text{ADVS}}\|_{\mathbf{Q}} = \sqrt{\boldsymbol{\eta}_{\text{ADVS}}^T \mathbf{Q} \boldsymbol{\eta}_{\text{ADVS}}}$ and $\|\bar{\mathbf{u}}\|_{\mathbf{R}} = \sqrt{\bar{\mathbf{u}}^T \mathbf{R} \bar{\mathbf{u}}}$. Here, $\mathbf{Q} \in \mathbb{R}^{(4n+8)\times(4n+8)}$ and $\mathbf{R} \in \mathbb{R}^{6\times 6}$ are positive definite matrices used to compute the weighted norms. Additionally, $\mathcal{X}$ and $\mathcal{U}$ represent the state and input constraints, respectively. The objective is to determine the optimal control effort $\bar{\mathbf{u}}[k]$ at the kth time step that minimizes the cost function $J(\boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}[k])$. The optimal control input is typically denoted as $\bar{\mathbf{u}}^*[k]$, while the corresponding optimal cost at the kth time step is represented as $J^*(\boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}^*[k])$. This optimization problem can be efficiently solved using nonlinear programming techniques, such as Interior Point Optimization (IP-OPT), which are readily available in open-source solvers.

While monotonically weighted NMPC helps relax stringent terminal constraints, it requires a sufficiently long prediction horizon to ensure stability when the states are far from equilibrium. Moreover, selecting an appropriate horizon length depends on prior knowledge. To address this,

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

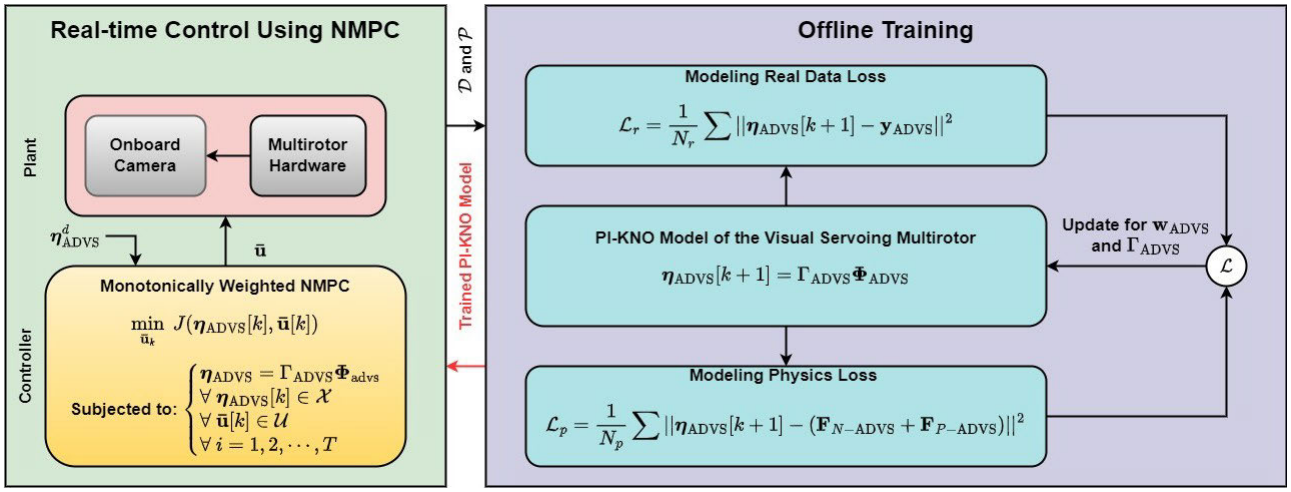


Fig. 2. Block diagram illustrating the overall PI-KNO control framework.

we propose an adaptive prediction horizon mechanism that dynamically adjusts based on system requirements, reducing computational effort when possible. At the kth time step, let $J^*(\boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}^*[k])$ and $J^*(\boldsymbol{\eta}_{\text{ADVS}}[k-1], \bar{\mathbf{u}}^*[k-1])$ denote the optimal costs at the kth and $(k-1)^{th}$ instances, respectively. The contraction factor κ_f is then computed as:

$$\kappa_f = \frac{J^*(\boldsymbol{\eta}_{\text{ADVS}}[k-1], \bar{\mathbf{u}}^*[k-1]) - J^*(\boldsymbol{\eta}_{\text{ADVS}}[k], \bar{\mathbf{u}}^*[k])}{J^*(\boldsymbol{\eta}_{\text{ADVS}}[k-1], \bar{\mathbf{u}}^*[k-1])} \tag{35}$$

A positive κ_f indicates cost contraction, while a negative value signifies cost expansion. The prediction horizon T is adaptively adjusted to maintain κ_f within a heuristic range $(\kappa_{min}, \kappa_{max})$, determined via simulations of model (33) and NMPC (34). The horizon update logic at each time step $k+1$ is defined as:

$$T_{k+1} = \begin{cases} T_k, & \text{if } \kappa_{min} \le \kappa_f \le \kappa_{max} \\ T_k - 1, & \text{if } \kappa_f > \kappa_{max} \ \&\ T_k > T_{min} \\ T_k + 1, & \text{if } \kappa_f < \kappa_{min} \ \&\ T_k < T_{max} \end{cases} \tag{36}$$

where T_k and T_{k+1} represent the prediction horizon at the kth and $(k+1)^{th}$ time steps, respectively. The minimum and maximum allowable horizon lengths, T_{min} and T_{max}, ensure feasibility, and this condition must hold for all $k \ge 2$. The stability analysis is presented in the supplementary material accompanying this paper.

V. Overall Framework

Fig. 2 illustrates the proposed PI-KNO control framework, which couples an offline training phase with a real-time NMPC-based control loop. The magenta-shaded region on the right represents the offline stage, where the PI-KNO model is trained using datasets $\mathcal{P}$ and $\mathcal{D}$ obtained from simulations and real flight experiments, respectively. Once trained, the model is integrated into the NMPC framework (green region on the left) as a surrogate for the multirotor dynamics to compute the optimal control input $\mathbf{u}^*_k$, which is executed on the hardware via the onboard computer to close the visual servoing loop. The newly collected flight data can be appended to $\mathcal{D}$ and $\mathcal{P}$ for retraining or fine-tuning, progressively enhancing the model's predictive accuracy and control performance over successive trials. Note that Fig. 2 depicts the functional structure of the proposed framework, while its real-time implementation workflow is detailed in Section VI.

VI. Results

This section is divided into two parts: Training and Benchmarking Studies and Hardware Implementation. The first evaluates the PI-KNO's performance under various conditions as an ablation study of the learning network, while the second presents the NMPC's performance when integrated with the trained PI-KNO on hardware. In both cases, the results are compared against state-of-the-art methods, with identical training datasets used across all experiments to ensure fairness.

A. Training and Benchmarking Studies

The training of the PI-KNO, along with the benchmarking algorithms, namely PINN [26] and the classical data-driven KNO (DD-KNO) [31], was performed on an NVIDIA A100 GPU. This GPU is equipped with 6,912 CUDA cores and 432 Tensor cores, enabling mixed-precision training with FP16 and TF32 to enhance computational efficiency. Its high memory bandwidth of 1.6 TB/s facilitates the efficient processing of large-scale neural network models, ensuring optimal performance during both training and benchmarking.

1) Training Overheads and Accuracy: The training configuration in this study utilizes a dataset of 5,000 points, evenly distributed between the physics dataset $\mathcal{P}$ (2,500 points) and the real-time dataset $\mathcal{D}$ (2,500 points). Both datasets are constructed based on four Points of Interest (POIs), represented as $\mathbf{r} \in \mathbb{R}^8$, which consequently determines the dimension of $\boldsymbol{\eta}_{\text{ADVS}}$ as $\mathbb{R}^{24}$. This, in turn, defines the dimension of Γ_{ADVS} as 24×6, leading to $\boldsymbol{\Phi}_{\text{ADVS}} = [\varphi_1 \cdots \varphi_6]^T$. The training parameters are set as follows: $N_{\text{Itr}} = 5,000$, $\zeta = 0.005$, $\delta_\Gamma = \delta_{\mathbf{w}} = 0.001$ which depicts the mean square error (MSE), and $d = 6$. The ratio of real-time dataset points to physics dataset points, referred to as data-skewness, is denoted by $\kappa = \frac{N_r}{N_p}$. It is important to note that the ratio κ implicitly

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

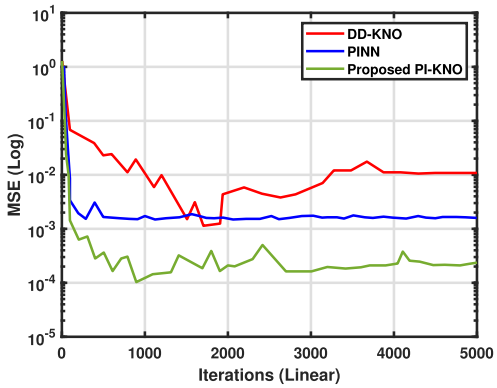


Fig. 3. Training accuracy v.s. Iterations for the Proposed PI-KNO, DD-KNO and the PINN algorithm.

determines the relative contribution of the data and physics losses within the total cost function $\mathcal{L} = \mathcal{L}_p + \mathcal{L}_r$. Under proportional batch sampling, the expected gradient of the combined loss can be expressed as:

$$\mathbb{E}[\nabla\mathcal{L}] \approx \frac{1}{1+\kappa}\nabla\mathcal{L}_p + \frac{\kappa}{1+\kappa}\nabla\mathcal{L}_r \tag{37}$$

where $\mathbb{E}[\cdot]$ represents the mathematical expectation, i.e. the expected value or mean of a random variable. The above equation indicates that the effective weighting factors of the physics and data components are $\omega_p \approx \frac{1}{1+\kappa}$ and $\omega_r \approx \frac{\kappa}{1+\kappa}$, respectively. Hence, decreasing κ increases the influence of the physics term, while larger κ biases learning toward data consistency. This formulation is mathematically equivalent to an explicit weighted loss of the form $\lambda_p\mathcal{L}_p + \lambda_r\mathcal{L}_r$ with $\lambda_p = \frac{1}{1+\kappa}$ and $\lambda_r = \frac{\kappa}{1+\kappa}$, but avoids introducing additional hyper-parameters by using dataset composition as the natural weighting mechanism. In this section, the data-skewness is set to $\kappa = 1$, ensuring an equal distribution of samples from $\mathcal{D}$ and $\mathcal{P}$ during training. The DD-KNO and the PINN are set to the specification presented in [26] and [31] respectively.

As shown in Fig. 3, the performance comparison of DD-KNO, PINN, and the proposed PI-KNO in terms of MSE over training iterations highlights the advantages of integrating physics-informed learning with Koopman operators. DD-KNO initially reduces error but remains highly sensitive to training data, leading to fluctuations and eventual stabilization at a relatively high MSE, reflecting its inability to generalize well due to the absence of physical constraints. PINN enforces physics-based consistency, ensuring stability throughout training, but its reliance on standard neural network architectures limits its capacity to efficiently capture complex nonlinear dynamics, resulting in slow convergence and an MSE of approximately 10^{-2}. In contrast, PI-KNO rapidly reduces error within the first 500 iterations, achieving an MSE below 10^{-4} and maintaining superior accuracy with minimal oscillations.

The superior convergence and lower MSE in Fig. 3 arise from the well-conditioned composite loss $\mathcal{L} = \mathcal{L}_r + \mathcal{L}_p$, which jointly optimizes the Koopman operator Γ and the lifting function $\mathbf{\Phi}$. In PI-KNO, $\mathcal{L}_r$ enforces trajectory agreement, while $\mathcal{L}_p = \|\mathbf{\Phi}_{k+1} - \Gamma\mathbf{\Phi}_k\|^2$ ensures physics consistency in the lifted space. In contrast, the data-driven KNO learns Γ and $\mathbf{\Phi}$ purely from empirical correlations, often overfitting local trajectory variations and producing an ill-conditioned, non-convex loss landscape. The inclusion of $\mathcal{L}_p$ regularizes learning by aligning gradient updates with physically admissible directions, reducing curvature anisotropy, and stabilizing optimization. While PINN also imposes physical residuals, they are applied in the nonlinear state space, where higher-order dependencies yield steep curvature and slower convergence. By enforcing a quadratic evolution constraint in the Koopman-lifted space, PI-KNO enables smoother gradient propagation, larger stable update steps, and superior convergence with lower MSE than both PINN and data-driven KNO.

TABLE I
TRAINING OVERHEAD (LATENCY) COMPARISON

Algorithm	Training Overhead (sec)
Proposed	**1024**
PINN [26]	1193
DD-KNO [31]	5438

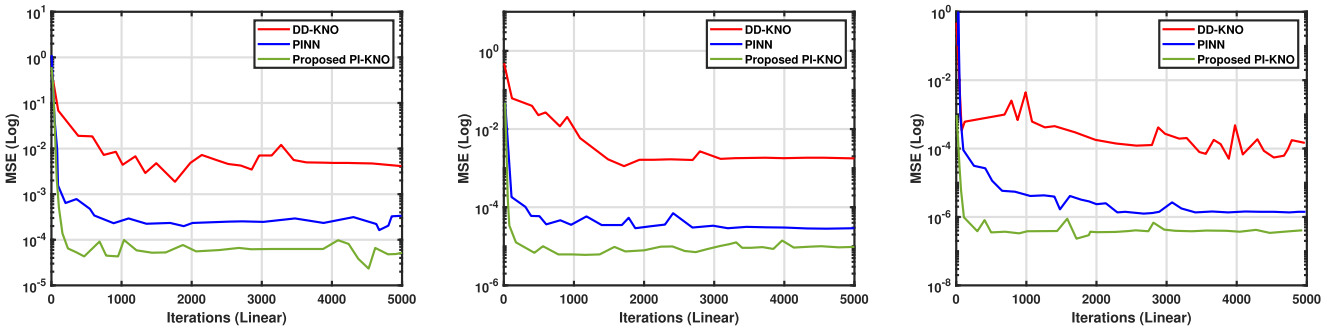
In addition to evaluating the MSE, this subsection also examines the training overhead, specifically the computational cost in terms of training time. Table I presents the training overhead (in seconds) for a data-skewness of 1. For PI-KNO, the training overhead is defined as the time elapsed from the start of training until the conditions $J_\Gamma < 0.01$ and $J_{\mathbf{w}} < 0.01$ are met. In the case of PINN and DD-KNO, the training overhead is measured as the time taken from the start of training until the total cost function converges to a value below 0.01. This tolerance is selected based on the observation in Fig. 3, where DD-KNO does not converge below a tolerance threshold of 0.01.

The training overhead results indicate that PI-KNO significantly improves computational efficiency, reducing training time by 14.16% compared to PINN and 81.17% compared to DD-KNO. This demonstrates the advantage of PI-KNO in accelerating convergence by integrating physics constraints with a structured learning approach. While PINN benefits from physics-based regularization, it lacks a dedicated transformation of system dynamics, resulting in a moderately higher computational cost. In contrast, DD-KNO, relying solely on data without any physics constraints, requires nearly 5 times the training time of PI-KNO, making it less effective for modeling complex nonlinear dynamics as in this application.

2) Effect of Varying Data-Skewness κ: This section examines the effect of data-skewness κ on the training accuracy of PI-KNO, PINN, and DD-KNO. The parameter κ is varied geometrically from 1 to $\frac{1}{64}$ with a common ratio of $\frac{1}{4}$. A balanced dataset ($\kappa = 1$) contains equal numbers of real-time and physics loss samples, whereas a highly skewed dataset ($\kappa = \frac{1}{64}$) includes far fewer real-time samples. Such imbalance can cause PI-KNO and PINN to misrepresent system dynamics, leading to larger tracking errors in real-time experiments.

As shown in Fig. 4, all three algorithms exhibit decreasing MSE with lower κ, demonstrating the regularizing influence of physics-based data. PI-KNO consistently achieves the lowest MSE across all κ, benefiting from its structured physics embedding. PINN follows a similar trend but with higher

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.



(a) $\kappa = \frac{1}{4}$ (b) $\kappa = \frac{1}{16}$ (c) $\kappa = \frac{1}{64}$

Fig. 4. Training accuracy v.s. Iterations for the Proposed, DD-KNO and the PINN algorithm for κ varying between $\frac{1}{4}$ and $\frac{1}{64}$.

TABLE II

TRAINING OVERHEAD (IN SEC) COMPARISON BETWEEN THE PROPOSED PI-KNO AND PINN FOR DIFFERENT κ VALUES

Algorithm	$\kappa = 1/1$	$\kappa = 1/4$	$\kappa = 1/16$	$\kappa = 1/64$
Proposed	**1024**	**653**	**338**	**128**
PINN	1193	724	426	157

residual error due to less efficient nonlinear representation, while DD-KNO remains least effective since it lacks explicit physics regularization. The reduction in MSE with decreasing κ reflects improved generalization through constrained optimization, which limits overfitting to limited real-time data. At extreme imbalance ($\kappa = \frac{1}{64}$), both PI-KNO and PINN show minor fluctuations, as insufficient real-time data reduce their ability to correct residual modeling inaccuracies. Thus, while increasing the proportion of physics loss data enhances convergence and stability, sufficient real-time data are still required for accurate adaptation to system dynamics. These results confirm that an optimal intermediate κ offers the best trade-off between stability and generalization during training.

Since κ primarily influences PINN and PI-KNO, their training overhead is compared in Table II. Unlike the training overhead analysis in Table I, which focuses on the time taken to reach a predefined error threshold, this comparison considers the total training time required for each algorithm to converge to the set tolerances. As seen from Table II, the training overhead for both PINN and PI-KNO decreases as κ is reduced, demonstrating the impact of data-skewness on convergence time. This reduction is attributed to the increasing contribution of physics loss data, which acts as an implicit regularizer, improving learning efficiency and reducing reliance on noisy real-time data. With lower κ, the model is guided more by analytically defined physics constraints, leading to smoother gradient updates and minimizing variability in real-time measurements. This shift accelerates convergence by reducing computational complexity per iteration. PI-KNO benefits more from this effect than PINN due to its structured learning approach, which enables more efficient feature extraction and improved generalization. Consequently, PI-KNO requires fewer iterations to reach a given accuracy threshold, making it, on average, 16.81% faster than PINN. This trend highlights the trade-off between real-time data reliance and physics-based supervision, where an increased emphasis on physics loss data enhances computational efficiency while maintaining model accuracy.

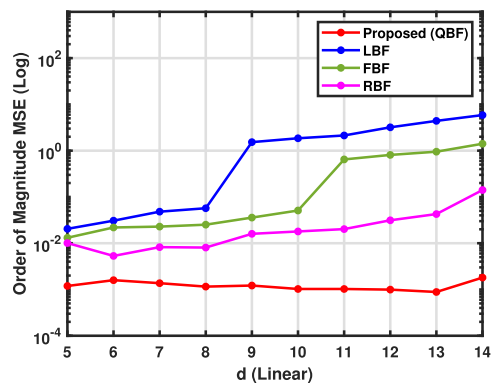


Fig. 5. Order of magnitude of training accuracy vs. d for different basis function variants in PI-KNO.

3) Effect of Varying d and φ: This subsection analyzes the influence of the number of basis functions d and the choice of basis family φ on PI-KNO performance. To isolate their effects, each attribute is varied independently while keeping the other fixed, under a data-skewness condition of $\kappa = 1$. Alongside the quadratic basis function (QBF) used in this study, three alternatives are considered: the linear basis function (LBF), Fourier basis function (FBF), and radial basis function (RBF). The performance, measured by the order of magnitude of the resulting MSE across varying d, is presented in Fig. 5. QBF consistently achieves the lowest MSE for all d, effectively capturing nonlinear multirotor dynamics without overfitting. LBF exhibits the highest error, as its linear mapping fails to represent nonlinear thrust-force coupling, aerodynamic effects, and attitude-dependent forces. FBF performs moderately at low d by modeling oscillatory behavior but deteriorates at higher d due to its inability to capture non-periodic transients such as rapid attitude changes and complex thrust variations. RBF, relying on localized kernels, outperforms LBF and FBF for small d yet degrades at large d because of overfitting and reduced generalization across flight conditions. Overall, QBF offers the best balance between accuracy and computational efficiency, capturing dominant nonlinearities while maintaining stability and generalization.

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

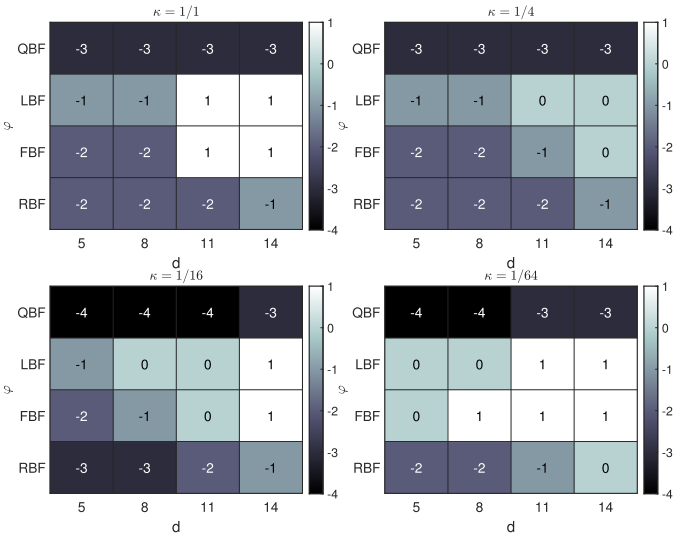
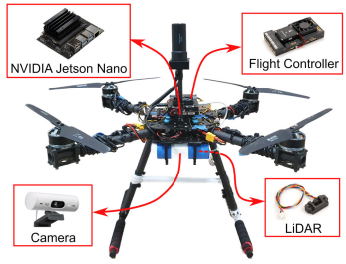


Fig. 6. Heat maps illustrating the order of magnitude of training error ($\log_{10}$(MSE)) across varying data-skewness κ, basis count d, and basis family φ for the proposed PI-KNO framework. Each cell represents the steady-state error exponent after convergence, highlighting the combined influence of data balance and model complexity on learning performance.

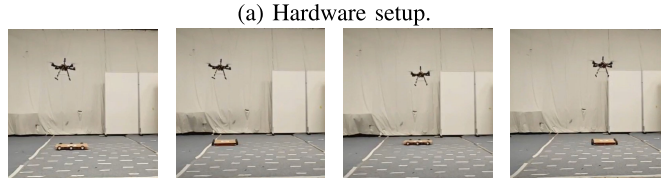
4) Joint Influence of Varying κ, d and φ: Fig. 6 illustrates the order of magnitude of the training error ($\log_{10}$(MSE)) obtained by the proposed PI-KNO framework across varying data-skewness ratios $\kappa \in \{1/1, 1/4, 1/16, 1/64\}$, basis counts $d \in \{5, 8, 11, 14\}$, and basis families $\varphi \in \{\text{QBF}, \text{LBF}, \text{FBF}, \text{RBF}\}$. Each cell represents the base-10 exponent of the steady-state MSE, capturing the combined influence of data balance and model complexity on learning performance. As observed earlier for $\kappa = 1$ (Fig. 5), QBF maintains a uniform MSE of about 10^{-3} for all d, whereas the remaining basis families show higher error magnitudes and stronger sensitivity to dimensionality.

As κ decreases from 1 to 1/4, the physics-loss weight $\omega_p \approx 1/(1+\kappa)$ increases, emphasizing physically consistent updates and improving conditioning. This reduces MSE by nearly one order of magnitude for most configurations. QBF sustains MSE $\approx 10^{-3}$ across all d, while RBF attains comparable accuracy for $d = 8$–11. FBF remains at 10^{-2}–10^{-1} due to oscillatory loss behavior, and LBF near 10^{0} because of its limited expressivity. At $\kappa = 1/16$, QBF and RBF improve further, reaching 10^{-4}–10^{-3} for moderate d, whereas FBF and LBF degrade owing to unstable gradients and under-representation of nonlinear dynamics.

Under extreme imbalance ($\kappa = 1/64$), the physics term dominates training. QBF remains stable with MSE $\in [10^{-4}, 10^{-3}]$, confirming its alignment with the smooth Koopman manifold. RBF begins to lose generalization at large d (MSE $\approx 10^{-1}$), while FBF and LBF diverge to 10^{0}–10^{1} due to amplified high-frequency and linear biases. Overall, decreasing κ enhances generalization up to an optimal range ($1/16 \le \kappa \le 1/4$), beyond which over-regularization limits data correction. Across all κ and d, QBF achieves the lowest and most consistent MSE, outperforming RBF by one–two orders and FBF/LBF by three–four, confirming its superior conditioning and convergence under both data-rich and physics-dominated regimes.



(a) Hardware setup.



(b) Experimental sequence for a square trajectory tracking.

Fig. 7. Experimental setup and trial sequence (video link [38]).

TABLE III
HARDWARE PARAMETERS

Parameters	Value	Parameters	Value
m	1.5 kg	g	9.8 m/sec^2
J_x	0.0150 kg.m^2	J_y	0.00150 kg.m^2
J_z	0.0300 kg.m^2	f	27 mm

B. Hardware Implementation

Based on the analysis in the previous section, the PI-KNO used for real-time experiments is selected with $d = 6$ and a quadratic basis function. In all subsequent results, the parameter κ is set to 1, and the trained PI-KNO is integrated with the monotonically weighted NMPC to control the ADVS multirotor model. The detailed hardware results are provided in the following subsections.

1) Hardware and Task Description: The effectiveness of the proposed approach is evaluated through trajectory tracking experiments on a quadrotor system, as shown in Fig. 7a. The quadrotor is built on a lightweight and durable Tarot Iron-man carbon-fiber frame. A Pixhawk 6X flight controller manages the lower-level control stack, using onboard sensors to estimate the system states and convert the computed control inputs $\bar{u}^*_{dep}$, $\bar{\mathbf{u}}^*_{pos}$, and $\bar{\mathbf{u}}^*_i$ into pulse-width modulated actuator commands. The upper-layer control stack implements the monotonically weighted NMPC with the PI-KNO model on an NVIDIA Jetson Nano running ROS 2. Communication between the Pixhawk 6X and the Jetson Nano is established via UART using MAVLink. Visual perception is provided by a downward-facing Logitech BRIO 500 camera with a 4K ultra-high-definition sensor, while a co-located LiDAR sensor enhances depth estimation and navigation accuracy. The hardware parameters are summarized in Table III.

Trajectory tracking via visual servoing is implemented using a moving autonomous ground vehicle (AGV), represented in the image plane by four points of interest (POIs), resulting in $\mathbf{r} \in \mathbb{R}^8$, along with its velocity in meters per second. For NMPC, a 1-second prediction horizon and a 1 kHz publishing frequency are selected, ensuring compatibility with

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

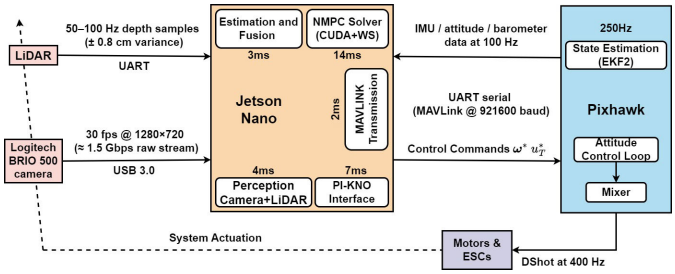


Fig. 8. Implementation workflow.

Pixhawk-based hardware. The Pixhawk's outer-loop operates at 50 Hz, while the inner attitude loop runs at 250 Hz. Additionally, the cost function matrices $\mathbf{Q} \in \mathbb{R}^{24\times 24}$ and $\mathbf{R} \in \mathbb{R}^{6\times 6}$ are chosen as positive definite, with the parameter p set to 3.

2) Implementation Workflow: The proposed PI-KNO-based augmented dynamics visual servoing (ADVS) framework is implemented on a Jetson–Pixhawk embedded system that performs perception, prediction, and control in a unified real-time loop (Fig. 8). The NVIDIA Jetson Nano executes visual perception, sensor fusion, and NMPC optimization, while the Pixhawk provides deterministic actuation through its internal mixer. Each control cycle begins with image acquisition from a downward-facing Logitech BRIO 500 camera (30 fps, USB 3.0), with frames processed via OpenCV in ROS 2 to detect four visual markers on the target. The pixel coordinates and LiDAR-based depth z_c are fused with inertial and attitude data received from the Pixhawk (100 Hz via MAVLink) to form the augmented state vector $\boldsymbol{\eta}_{\text{ADVS}}$. Perception, fusion, and state construction complete within 4–5 ms per cycle. The trained PI-KNO network operates in inference mode using PyTorch with CUDA acceleration to predict the lifted-state evolution over a finite horizon. The NMPC layer, implemented using the `acados` solver, minimizes the composite physics-informed cost to compute the optimal input $\bar{u}^*[k]$, yielding an overall decision latency of approximately 22–23 ms per control cycle.

The computed control vector $\bar{u}^*[k]$ containing total thrust and body moments is mapped to the Pixhawk command space and transmitted via UART (921600 baud) using the `SET_ATTITUDE_TARGET` MAVLink protocol. The Pixhawk executes its 400 Hz actuator loop, converting attitude-rate and thrust references into rotor-level thrusts. The overall perception-to-actuation latency is 31–34 ms, corresponding to a 30 Hz effective control rate - well within PX4 Offboard real-time limits. The hybrid camera - LiDAR localization provides robust depth observability, with an average reprojection residual within ± 2 pixels ($\approx$ 1.5 cm at 1.2 m altitude) and LiDAR depth variance below ± 0.8 cm. These uncertainties are modeled as part of the aggregated disturbance term D_{ADVS}, enabling the NMPC to account for their effects within its predictive cost. This compact perception–control pipeline demonstrates that the proposed PI-KNO-based NMPC achieves accurate, depth-consistent visual servoing of quadrotors in real time under embedded hardware constraints. A detailed description of the implementation workflow, timing breakdown, and system architecture is provided in the supplementary material accompanying this manuscript.

TABLE IV
AVERAGE %SDTW ERRORS FOR THE PROPOSED PI-KNO, PINN AND DD-KNO APPROACH WITH A 70.83% UNCERTAINTY IN CAMERA PARAMETERS

Trajectory	Square	Lemniscate
Proposed	**75.66**	**46.04**
PINN	87.76	50.22
DD-KNO	105.97	77.34

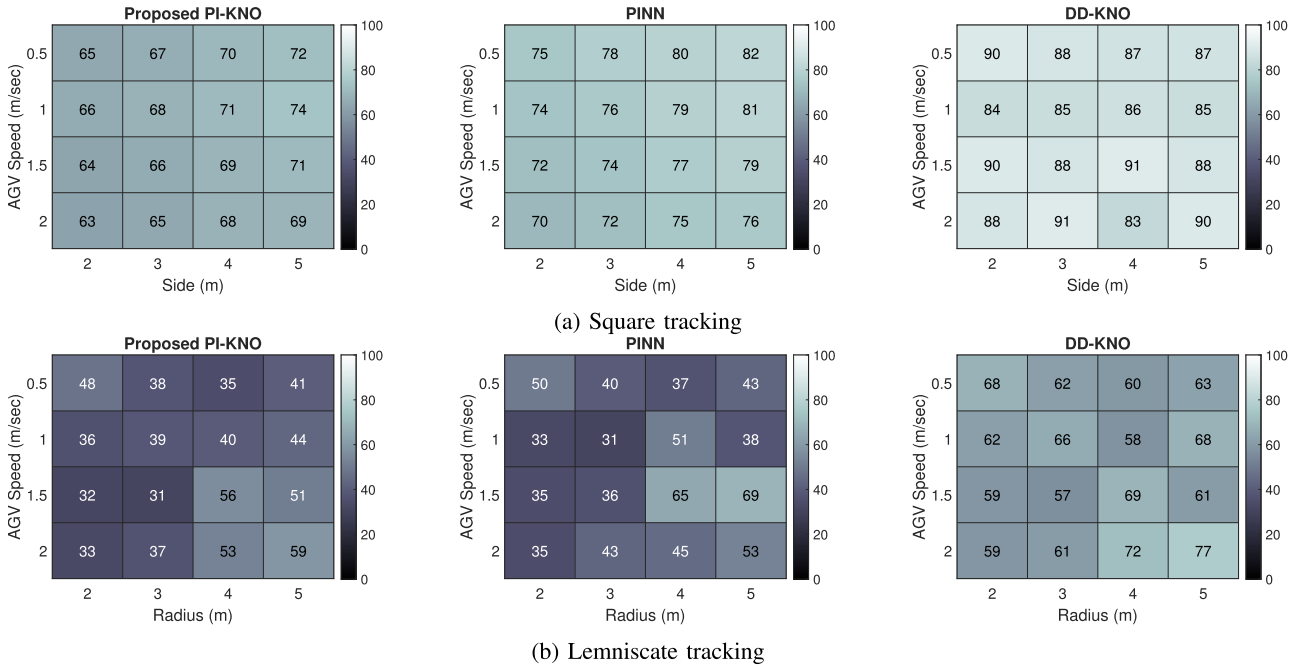
3) Tracking Accuracy Benchmarking: In this paper, the quadrotor's tracking performance is evaluated using two reference trajectories: a lemniscate and a square path. The lemniscate trajectory, with its continuous curvature and smooth transitions, assesses the controller's ability to follow dynamic paths with varying acceleration demands. The square trajectory, featuring sharp turns and abrupt directional changes, tests responsiveness to rapid reorientation. Fig. 7 illustrates an experimental sequence of the quadrotor tracking the AGV along a square trajectory, with the corresponding experimental video available in [38].

Tracking accuracy is evaluated using Percentage Soft Dynamic Time Warping (%SDTW) [39], a normalized, differentiable variant of DTW that replaces the hard-min operator with a soft-min, enabling smooth evaluation in learning-based settings. The metric is normalized against a baseline PID-controlled trajectory, ensuring fair comparison across control schemes. A comparative %SDTW analysis is conducted for PI-KNO, PINN, and DD-KNO integrated with monotonically weighted NMPC, over AGV speeds of 0.5–2 m/s and lemniscate radii and square side lengths of 2–5 m.

Fig. 9 presents the %SDTW variations for both trajectories, where a lower value indicates better tracking. The proposed PI-KNO consistently achieves the lowest %SDTW, averaging 68% (square) and 42.06% (lemniscate), compared to 76.25% and 44% for PINN, and 87.56% and 63.87% for DD-KNO, respectively. This corresponds to reductions of 8.25% and 19.56% over PINN and DD-KNO for the square trajectory, and 1.94% and 21.81% for the lemniscate trajectory, demonstrating the superior tracking capability of the proposed PI-KNO.

4) Handling Parametric Uncertainties: Since camera uncertainties are incorporated into the physics-loss dataset, the proposed PI-KNO maintains reliable tracking alongside the monotonically weighted NMPC as long as data skewness remains close to unity. To assess robustness, the tracking task is repeated using a camera with 70.38% uncertainty in its intrinsic parameters. The average %SDTW for PI-KNO, PINN, and DD-KNO is summarized in Table IV. The proposed method achieves 10.6% and 22.2% lower %SDTW than PINN and DD-KNO, respectively, for the square trajectory, and 11% and 36.4% lower for the lemniscate trajectory, confirming superior tracking accuracy even under camera parameter uncertainties. Furthermore, PI-KNO exhibits smaller variations in %SDTW, both with and without uncertainty, demonstrating consistent performance. For the square and lemniscate trajectories, PI-KNO shows an increase of only 11.27% and 9.48% in %SDTW when uncertainties are introduced, compared to

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.



(a) Square tracking

(b) Lemniscate tracking

Fig. 9. Heatmap of %SDTW between the proposed approach, the PINN and the DD-KNO acting alongside the monotonically weighted NMPC when tracking a square and lemniscate trajectory at varying speeds and widths in real-time.

TABLE V

BASELINE NMPC PER-ITERATION LATENCY (WITHOUT WARM-STARTING OR CUDA ACCELERATION) FOR PI-KNO, PINN, AND DD-KNO ON JETSON NANO

Algorithm	Baseline Latency (ms)	Standard Deviation (ms)
Proposed	**48.11**	**3.89**
PINN	50.35	4.21
DD-KNO	37.21	3.04

TABLE VI

NMPC PER-ITERATION LATENCY COMPARISON FOR PI-KNO, PINN, AND DD-KNO WITH WARM-STARTING (WS) AND CUDA ACCELERATION ON JETSON NANO

Algorithm	WS (ms)	CUDA (ms)	CUDA + WS (ms)
Proposed	**38.78**	**35.90**	**31.51**
PINN	40.74	37.69	35.19
DD-KNO	29.62	27.69	25.36

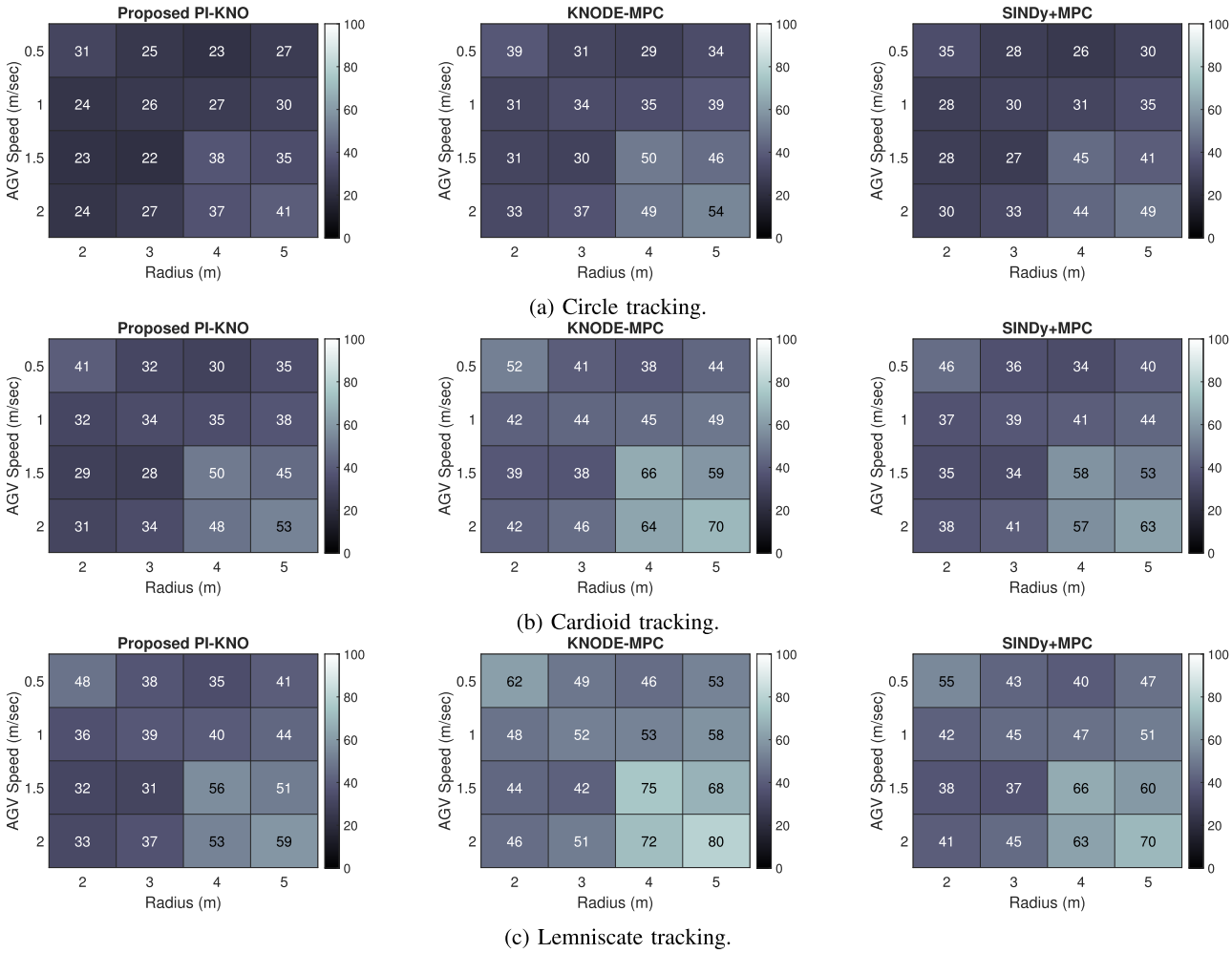
15.10% and 14.13% for PINN, and 21.03% and 19.55% for DD-KNO. These results highlight the superior robustness of the proposed approach in handling sensor-induced variations.

5) Computational Efficiency and Real-Time Feasibility Benchmarking: This subsection evaluates the real-time latency of the monotonically weighted NMPC integrated with the trained PI-KNO, PINN, and DD-KNO models. All networks are trained offline, and only their fixed weights are used during online deployment. The measured latency therefore corresponds to the combined inference and NMPC optimization time per control cycle on the Jetson Nano, excluding offline training overhead. The per-iteration latency t_{iter} is defined as $t_{\text{iter}} = t_{\text{out}} - t_{\text{in}}$, where t_{in} and t_{out} denote the timestamps for feedback receipt and control output transmission, respectively. The mean and standard deviation of t_{iter} are obtained from 500 NMPC updates. Table V lists the baseline per-iteration times without optimization. PI-KNO and PINN exhibit similar latencies of 48.11 ms and 50.35 ms, while DD-KNO achieves 37.21 ms due to its smaller network size but requires more solver iterations because of poorer conditioning, resulting in a longer overall control cycle.

With warm-starting (WS) and CUDA acceleration, the computation time reduces significantly, as shown in Table VI. Warm-starting decreases Newton iterations N_{iter} by 19–22%, and GPU acceleration reduces Jacobian/Hessian evaluation cost by 23–28%, yielding a total latency reduction of 30–40%. Under this optimized configuration, PI-KNO achieves a per-iteration latency of 31.51 ms, compared to 35.19 ms for PINN and 25.36 ms for DD-KNO. Although DD-KNO has the smallest iteration time, it typically requires about 15 solver iterations to converge, versus 11 for PINN and only 7 for PI-KNO. The resulting total control-cycle times are $T_{\text{cycle}}^{\text{PI-KNO}} = 220.6$ ms, $T_{\text{cycle}}^{\text{PINN}} = 387.1$ ms, and $T_{\text{cycle}}^{\text{DD-KNO}} = 380.4$ ms, corresponding to update rates of 4.5 Hz, 2.6 Hz, and 2.6 Hz, respectively. The improved conditioning of PI-KNO's Koopman-linearized dynamics yields faster convergence with fewer iterations, enabling real-time implementation.

During deployment, the Jetson Nano transmits the optimized control vector $\mathbf{u}_k^*$ to the Pixhawk via MAVLink (921600 baud), while the Pixhawk executes its 250–400 Hz inner control loops and processes offboard updates at up to 50 Hz. With the practical NMPC rate of 4–5 Hz, the proposed PI-KNO fully satisfies real-time constraints, as the serial communication delay (2–4 ms) is negligible compared to computation time. Overall, PI-KNO achieves up to 40% reduction in latency relative to the unoptimized baseline while maintaining high control precision, confirming its suitability for onboard deployment in resource-constrained aerial platforms. A detailed analysis of

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.



(a) Circle tracking.

(b) Cardioid tracking.

(c) Lemniscate tracking.

Fig. 10. Heatmap of %SDTW between Proposed PI-KNO+NMPC, KNODE-MPC [34] and SINDy+MPC [35] approach when tracking a circle, cardioid and lemniscate trajectory at varying speeds and radius.

the latency breakdown, optimization steps, and convergence behavior is provided in the supplementary material of this manuscript.

6) Comparison With SOTA Control Techniques: To assess robustness and generalization, the proposed PI-KNO framework is evaluated on three trajectories of increasing complexity, namely: circle, cardioid, and lemniscate. The circle represents uniform curvature and steady angular motion, the cardioid introduces a cusp causing transient thrust variations, and the lemniscate combines alternating curvature and direction reversals, forming a comprehensive benchmark for dynamic learning and closed-loop control.

A first comparison is made with the Sparse Identification of Nonlinear Dynamics (SINDy) framework [36], which identifies dominant nonlinear terms from a candidate library for use in model predictive control [35]. While SINDy+MPC is data-efficient, its accuracy depends heavily on the chosen library and excitation coverage of the training data. The proposed PI-KNO overcomes this by embedding physics directly in the learning process, allowing the Koopman operator to remain stable and generalize under unobserved curvatures, particularly beneficial for complex trajectories such as the cardioid and lemniscate. The second comparison involves the Knowledge-based Neural Ordinary Differential Equation (KNODE-MPC) [34], which augments a nominal physical model with a learned residual. Although it captures unmodeled effects, its performance deteriorates when data are sparse or unevenly distributed. By contrast, PI-KNO constrains the learning process via a physics-informed loss, achieving faster convergence and improved stability even with limited data.

TABLE VII

AVERAGE %SDTW FOR PROPOSED PI-KNO, KNODE-MPC [34] AND SINDY+MPC [35] ACROSS CIRCLE, CARDIOID, AND LEMNISCATE TRAJECTORIES

Controller	Circle	Cardioid	Lemniscate
PI-KNO (Proposed)	**28.75 %**	**37.19 %**	**42.06 %**
KNODE-MPC [34]	37.63 %	48.69 %	56.19 %
SINDy+MPC [35]	33.75 %	43.50 %	49.38 %

The average tracking accuracy, quantified using %SDTW, is summarized in Table VII. PI-KNO consistently achieves the lowest error, 28.75%, 37.19%, and 42.06% for the circle, cardioid, and lemniscate trajectories, respectively, outperforming KNODE-MPC (37.63%, 48.69%, 56.19%) and SINDy+MPC (33.75%, 43.50%, 49.38%). As trajectory

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

complexity increases, all methods show rising %SDTW, but the relative ranking remains unchanged, confirming PI-KNO's superior robustness and generalization across nonlinear motion profiles. The detailed heatmap analysis of trajectory-dependent performance is shown in Fig. 10.

VII. Conclusion

This paper presented a Physics-Informed Koopman Neural Operator (PI-KNO) for augmented dynamics visual servoing of multirotors. By integrating Koopman operator theory with physics-informed neural learning, the framework provides a structured and data-efficient means to model complex nonlinear dynamics while retaining physical interpretability. The learned Koopman representation is embedded within a monotonically weighted NMPC scheme that adaptively adjusts the prediction horizon and enforces input and state constraints for stable real-time trajectory tracking. Extensive simulation and hardware experiments benchmarked the proposed approach against DD-KNO, PINN, KNODE-MPC [34], and SINDy+MPC [35], [36]. Across circular, cardioid, and lemniscate trajectories, PI-KNO achieved the lowest %SDTW, outperforming SINDy+MPC by 14.7%, KNODE-MPC by 24.2%, PINN by 40.0%, and DD-KNO by 52.4%, with a 16.81% reduction in training time and a 19.56% improvement in tracking accuracy. Real-world experiments on a Jetson-Pixhawk platform confirmed robust performance, maintaining accurate trajectory alignment even under 70.83% uncertainty in camera calibration. These results indicate that physics embedding in PI-KNO improves convergence robustness by reducing error propagation compared to data-driven and sparse models. Future work will investigate online adaptive Koopman learning and cooperative multi-agent control using shared representations.

References

[1] M. W. Spong, S. Hutchinson, and M. Vidyasagar, *Robot Modeling Control*. Hoboken, NJ, USA: Wiley, 2020.

[2] S. Doncieux and J.-B. Mouret, "Beyond black-box optimization: A review of selective pressures for evolutionary robotics," *Evol. Intell.*, vol. 7, no. 2, pp. 71–93, Aug. 2014.

[3] M. Feroskhan and T. H. Go, "Control strategy of sideslip perching maneuver under dynamic stall influence," *Aerosp. Sci. Technol.*, vol. 72, pp. 150–163, Jan. 2018.

[4] A. K. Kamath and M. Feroskhan, "Addressing target loss and actuator saturation in visual servoing of multirotors: A nonrecursive augmented dynamics control approach," *Nonlinear Eng.*, vol. 13, no. 1, Aug. 2024, Art. no. 20240018.

[5] M. Xu, A. Hu, and H. Wang, "Image-based visual impedance force control for contact aerial manipulation," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 1, pp. 518–527, Jan. 2023.

[6] H. Yu, Z. Zhang, T. Pei, J. Han, Y. Fang, and X. Liang, "Visual servoing-based anti-swing control of cable-suspended aerial transportation systems with variable-length cable," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 5955–5965, 2025.

[7] I. Lopez-Sanchez and J. Moreno-Valenzuela, "PID control of quadrotor UAVs: A survey," *Annu. Rev. Control*, vol. 56, Jan. 2023, Art. no. 100900.

[8] S. Khatoon, D. Gupta, and L. K. Das, "PID & LQR control for a quadrotor: Modeling and simulation," in *Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI)*, Sep. 2014, pp. 796–802.

[9] J. Madeiras, C. Cardeira, and P. Oliveira, "Trajectory tracking of a quadrotor via full-state feedback control," *Aerosp. Sci. Technol.*, vol. 147, Apr. 2024, Art. no. 109013.

[10] A. K. Kamath, Z. Zheng, and M. Feroskhan, "Depth-independent augmented dynamics visual servoing of multirotors," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 61, no. 2, pp. 5444–5453, Apr. 2025.

[11] B. Yan, P. Shi, and C.-C. Lim, "Robust formation control for nonlinear heterogeneous multiagent systems based on adaptive event-triggered strategy," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 4, pp. 2788–2800, Oct. 2022.

[12] L. Grüne, J. Pannek, L. Grüne, and J. Pannek, *Nonlinear Model Predictive Control*. Cham, Switzerland: Springer, 2017.

[13] H. Merabti, I. Bouchachi, and K. Belarbi, "Nonlinear model predictive control of quadcopter," in *Proc. 16th Int. Conf. Sci. Techn. Autom. Control Comput. Eng. (STA)*, Dec. 2015, pp. 208–211.

[14] K. Zhang, Y. Shi, and H. Sheng, "Robust nonlinear model predictive control based visual servoing of quadrotor UAVs," *IEEE/ASME Trans. Mechatronics*, vol. 26, no. 2, pp. 700–708, Apr. 2021.

[15] X. Ping, J. Yao, B. Ding, and Z. Li, "Tube-based output feedback robust MPC for LPV systems with scaled terminal constraint sets," *IEEE Trans. Cybern.*, vol. 52, no. 8, pp. 7563–7576, Aug. 2022, doi: 10.1109/TCYB.2020.3041334.

[16] M. Yuan, Y. Wang, L. Li, T. Chai, and W. T. Ang, "Safety-based speed control of a wheelchair using robust adaptive model predictive control," *IEEE Trans. Cybern.*, vol. 54, no. 8, pp. 4464–4474, Aug. 2024, doi: 10.1109/TCYB.2023.3309369.

[17] J. Qiao, J. Sun, and X. Meng, "Event-triggered adaptive model predictive control of oxygen content for municipal solid waste incineration process," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 1, pp. 463–474, Jan. 2024.

[18] Y. W. Kerk, K. M. Tay, C. H. Jong, and C. P. Lim, "On ordered weighted averaging operator and monotone Takagi–Sugeno–Kang fuzzy inference systems," *IEEE Trans. Cybern.*, vol. 55, no. 4, pp. 1540–1553, Apr. 2025.

[19] P. Shi, W. Sun, and X. Yang, "RBF neural network-based adaptive robust synchronization control of dual drive gantry stage with rotational coupling dynamics," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 2, pp. 1059–1068, Apr. 2023.

[20] N. Mohajerin, M. Mozifian, and S. Waslander, "Deep learning a quadrotor dynamic model for multi-step prediction," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2018, pp. 2454–2459.

[21] X. Zhang, Z. Hu, Y. Wang, F. Guo, Z. Li, and C.-Y. Su, "Adaptive neural control for hysteretic nonlinear systems with hysteresis neural direct inverse compensator and its application," *IEEE Trans. Cybern.*, vol. 54, no. 10, pp. 6230–6243, Oct. 2024.

[22] S. Bansal, A. K. Akametalu, F. J. Jiang, F. Laine, and C. J. Tomlin, "Learning quadrotor dynamics using neural network for flight control," in *Proc. IEEE 55th Conf. Decis. Control (CDC)*, Dec. 2016, pp. 4653–4660.

[23] S. Stebler, M. Campobasso, K. Kidambi, W. MacKunis, and M. Reyhanoglu, "Dynamic neural network-based sliding mode estimation of quadrotor systems," in *Proc. Amer. Control Conf. (ACC)*, May 2017, pp. 2600–2605.

[24] G. Yu, J. Reis, and C. Silvestre, "Quadrotor neural network adaptive control: Design and experimental validation," *IEEE Robot. Autom. Lett.*, vol. 8, no. 5, pp. 2574–2581, May 2023.

[25] W. Yang, Z. Li, G. Li, and L. Xu, "Multicontact safety-critical planning and adaptive neural control of a soft exosuit over different terrains," *IEEE Trans. Cybern.*, vol. 55, no. 5, pp. 2341–2354, May 2025.

[26] A. K. Kamath, S. G. Anavatti, and M. Feroskhan, "A physics-informed neural network approach to augmented dynamics visual servoing of multirotors," *IEEE Trans. Cybern.*, vol. 54, no. 11, pp. 6319–6332, Nov. 2024.

[27] S. Cai, Z. Mao, Z. Wang, M. Yin, and G. E. Karniadakis, "Physics-informed neural networks (PINNs) for fluid mechanics: A review," *Acta Mechanica Sinica*, vol. 37, no. 12, pp. 1727–1738, Dec. 2021.

[28] S. Sanyal and K. Roy, "RAMP-Net: A robust adaptive MPC for quadrotors via physics-informed neural network," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, May 2023, pp. 1019–1025.

[29] D. Bianchi, N. Epicoco, M. Di Ferdinando, S. Di Gennaro, and P. Pepe, "Physics-informed neural networks for unmanned aerial vehicle system estimation," *Drones*, vol. 8, no. 12, p. 716, Nov. 2024.

[30] Y. Liu, A. Sholokhov, H. Mansour, and S. Nabi, "Physics-informed Koopman network for time-series prediction of dynamical systems," in *Proc. ICLR Workshop AI4DifferentialEquations Sci.*, 2024. [Online]. Available: https://openreview.net/forum?id=mwHL5nOqOM

[31] W. Xiong, X. Huang, Z. Zhang, R. Deng, P. Sun, and Y. Tian, "Koopman neural operator as a mesh-free solver of non-linear partial differential equations," *J. Comput. Phys.*, vol. 513, Sep. 2024, Art. no. 113194.

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.

[32] D. Meng, Y. Zhu, J. Wang, and Y. Shi, "Koopman neural operator approach to fast flow prediction of airfoil transonic buffet," *Phys. Fluids*, vol. 36, no. 7, Jul. 2024, Art. no. 075182, doi: 10.1063/5.0217336. [Online]. Available: https://pubs.aip.org/aip/pof/article-pdf/doi/10.1063/5.0217336/20069575/075182_1_5.0217336.pdf

[33] Y. Xiao, X. Zhang, X. Xu, X. Liu, and J. Liu, "Deep neural networks with Koopman operators for modeling and control of autonomous vehicles," *IEEE Trans. Intell. Vehicles*, vol. 8, no. 1, pp. 135–146, Jan. 2023.

[34] K. Y. Chee, T. Z. Jiahao, and M. A. Hsieh, "KNODE-MPC: A knowledge-based data-driven predictive control framework for aerial robots," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2819–2826, Apr. 2022.

[35] U. Fasel, E. Kaiser, J. N. Kutz, B. W. Brunton, and S. L. Brunton, "SINDy with control: A tutorial," in *Proc. IEEE Conf. Decis. Control*, Dec. 2021, pp. 16–21.

[36] S. L. Brunton, J. L. Proctor, and J. N. Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems," *Proc. Nat. Acad. Sci. USA*, vol. 113, no. 15, pp. 3932–3937, Apr. 2016.

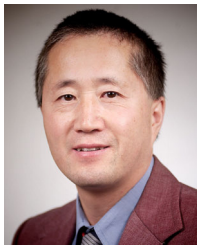
[37] Y. Lu and J. Lu, "A universal approximation theorem of deep neural networks for expressing probability distributions," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 3094–3105.

[38] A. K. Kamath, B. Yan, P. Shi, and M. Feroskhan. (2026). *Experimental Video: Real-Time Visual Servoing Using PI-KNO*. [Online]. Available: https://youtu.be/UW7hPILj4Qs

[39] M. Cuturi and M. Blondel, "Soft-DTW: A differentiable loss function for time-series," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 894–903.

Archit Krishna Kamath (Member, IEEE) received the bachelor's degree in electrical and electronics engineering from the M. S. Ramaiah Institute of Technology, Bengaluru, Karnataka, India, in 2015, and the master's and Ph.D. degrees from Indian Institute of Technology Kanpur (IIT Kanpur) in 2018 and 2023, respectively, with a specializing in control and automation. He is currently a Research Fellow with the Intelligent Cybernetics Laboratory, School of Mechanical and Aerospace Engineering, NTU Singapore. His research pursuits encompass various areas, including modeling and sliding mode control of both conventional and over-actuated quadrotors, utilization of physics-informed neural networks and graph neural networks in aerial robot control, and decentralized control of drone swarms. He is a TC Member of the Robotics and Intelligent Sensing Technical Committee on the IEEE SMC Society.

Bing Yan (Member, IEEE) received the B.Sc. degree in automation and the M.Sc. degree in control theory and control engineering from Northwestern Polytechnical University, Xi'an, China, in 2012 and 2015, respectively, and the Ph.D. degree in electrical and electronic engineering from The University of Adelaide, Adelaide, SA, Australia, in 2022. She is currently a Lecturer with The University of Adelaide. Her research interests include formation control, reinforcement learning, robust control, and multi-agent systems.



Peng Shi (Life Fellow, IEEE) is currently a Professor with The University of Adelaide. His research interests include systems and control theory and applications to autonomous and robotic systems, multi-agent systems, and cyber-physical systems. He is a member of the Academy of Europe and a fellow of Australian Academy of Technological Sciences and Engineering, Institute of Engineering and Technology, and Institution of Engineers Australia. He serves as the Editor-in-Chief for IEEE TRANSACTIONS ON CYBERNETICS, a Senior Editor for IEEE ACCESS, and an Distinguished Lecturer for IEEE Systems, Man, and Cybernetics Society.

Mir Feroskhan (Member, IEEE) received the bachelor's degree (Hons.) in aerospace engineering from Nanyang Technological University (NTU), Singapore, in 2011, and the Ph.D. degree in aerospace engineering from Florida Institute of Technology, USA, in 2016. Currently, he holds the position of an Assistant Professor with the School of Mechanical and Aerospace Engineering, NTU Singapore, and leads the Intelligent Cybernetics Group. Within this role, he conducts research in diverse areas, such as modeling and controlling avian-inspired drones, applying artificial intelligence to aerospace engineering, exploring multi-agent systems, addressing urban air mobility challenges, and advancing space situational awareness. He is an Active Member of IEEE and holds a position as a Technical Committee Member of the Robotics and Intelligent Sensing Technical Committee with the IEEE SMC Society. He received notable accolades, including the A*STAR's Young Individual Research Grant in 2022 for pioneering AI-driven solutions for UAV applications and the Dr. S. K. Leung Excellence Award in 2023 in recognition of his exemplary teaching activities. Additionally, he serves on the Editorial Board for Nature *Scientific Reports* and an Associate Editor for IEEE TRANSACTIONS ON CYBERNETICS.

Authorized licensed use limited to: J.R.D. Tata Memorial Library Indian Institute of Science Bengaluru. Downloaded on February 27,2026 at 04:47:04 UTC from IEEE Xplore. Restrictions apply.